
All Roads Lead to Rome: Flow-driven Multi-Anchor Exploration for Open-Environment Active 3D Mapping

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 To advance the development of embodied intelligence, Open-Environment Active
2 3D Mapping has attracted increasing attention, aiming to perform a long-horizon
3 and shortest trajectory exploration for reconstructing unseen scenarios. Since
4 only limited information about unseen environments is available, methods built
5 on the closed-set assumption, i.e., assuming that the test environments are similar
6 to those seen during training, cannot generalize satisfactorily. In existing active
7 mapping methods, long-horizon exploration is often guided by predicting a coarse
8 long-range goal and then converting it into an executable path. However, this stage
9 is usually formulated as single-point prediction. Under partial observability, the
10 same local observation may correspond to multiple plausible exploration direc-
11 tions, making such deterministic prediction prone to brittle decisions and degraded
12 performance in unseen scenarios. Our experiments further verify that this is a key
13 factor underlying their weak generalization. To address this issue, we reformulate
14 long-horizon target prediction as conditional multimodal anchor generation using
15 Conditional Flow Matching. Instead of predicting a single goal, our method learns
16 a conditional distribution over coarse exploration anchors from the current mapping
17 state. These anchors are first converted into executable candidate paths through
18 obstacle-aware planning. We then apply exploration-mode clustering to compress
19 geometrically similar trajectories and reduce candidate redundancy. Finally, a hier-
20 archical selection module selects the most promising mode and reranks paths within
21 it to produce the final executable trajectory. Experiments show that our method
22 improves generalization and reconstruction efficiency in open environments.

23 1 Introduction

24 Active 3D Mapping is a core problem in robotics and digital-twin applications, where an embodied
25 agent must actively move, acquire observations, and reconstruct the geometry of an unknown scene
26 as efficiently as possible. Unlike passive reconstruction [1, 2], active mapping [3, 4] requires not only
27 accurate surface reconstruction but also effective exploration decisions under a limited sensing budget.
28 Thus, the key challenge lies not only in *how to reconstruct*, but also in *how to plan*. Early works [5–7]
29 are typically built on the closed-set assumption, i.e., assuming that the test environments are similar
30 to those seen during training. However, in open environments, only limited information about unseen
31 scenes is available at test time, which makes such methods difficult to generalize satisfactorily.

32 Recently, open-environment benchmarks [8, 9] have extended active 3D mapping to cross-scene
33 and cross-difficulty settings, placing much stronger demands on generalization. In these settings, long-
34 horizon exploration is often guided by first predicting a coarse long-range goal. However, existing
35 methods on these benchmarks [8, 9] still largely formulate this stage as single-point prediction. This
36 formulation is fundamentally limited under partial observability, where the same local observation
37 may admit multiple plausible long-horizon exploration directions. As shown in Fig. 1, single-point

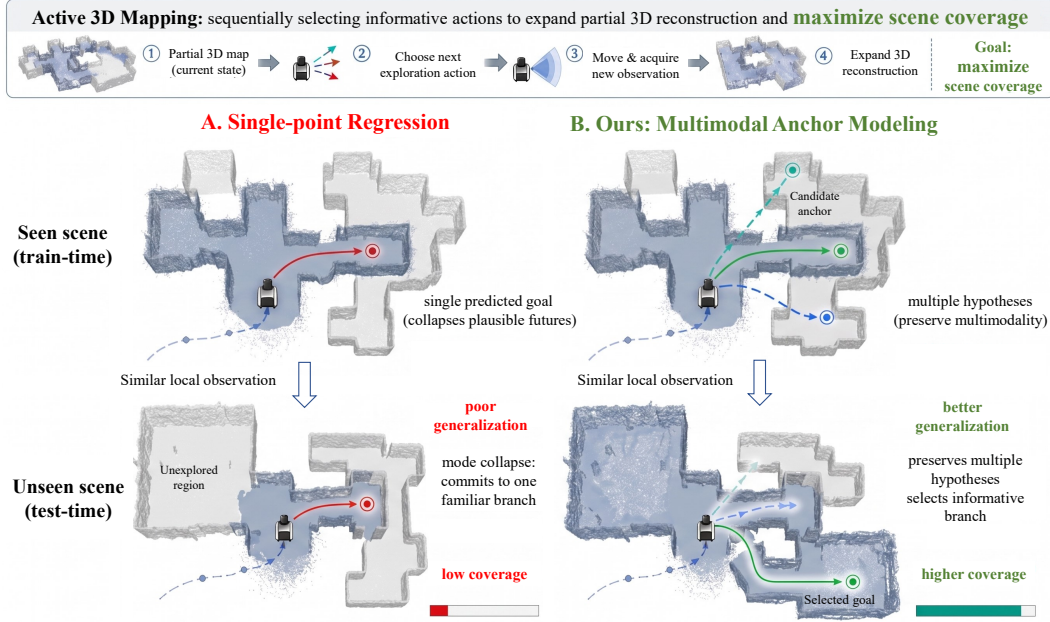


Figure 1: **Motivation.** Under partial observability, a similar local observation may support multiple plausible long-horizon exploration directions. Single-goal prediction collapses these futures into one dominant branch and generalizes poorly when the unseen scene continuation changes. In contrast, multimodal anchor modeling preserves alternative hypotheses before downstream planning, leading to better generalization and higher coverage.

38 regression collapses multiple plausible long-horizon futures into one dominant goal. When the unseen
 39 scene continuation changes at test time, it tends to commit to a familiar branch, leading to brittle
 40 decisions and weak generalization. Our experiments in Sec. C.2 further indicate that this single-goal
 41 prediction paradigm is a key factor underlying the weak generalization of existing methods.

42 This observation motivates us to move beyond single-goal prediction and instead model long-
 43 horizon exploration with multiple plausible goal hypotheses. Recent progress in generative model-
 44 ing [10, 11] suggests that conditional generation is particularly effective for representing multimodal
 45 futures. In particular, Conditional Flow Matching (CFM) [10] learns a continuous transport from a
 46 simple prior to a conditional target distribution, making it both stable and efficient for multimodal
 47 modeling. However, existing flow-based models in embodied decision-making tasks [12–15] are
 48 primarily designed for high-dimensional trajectory or action generation, whereas active 3D mapping
 49 fundamentally requires modeling long-horizon exploration intent under partial observability rather
 50 than directly synthesizing full behaviors.

51 To this end, we reformulate long-horizon target prediction in active 3D mapping as a conditional
 52 multimodal proposal generation problem, and adapt Conditional Flow Matching (CFM) to this setting
 53 in a task-specific manner so as to model multiple plausible long-horizon exploration hypotheses in a
 54 low-dimensional anchor space. The sampled anchors are then converted into executable candidate
 55 paths through obstacle-aware planning, compressed into exploration modes by anchor-guided cluster-
 56 ing, and finally resolved by a hierarchical selection module to produce the final trajectory. Beyond the
 57 architectural design, we provide theoretical analysis showing why deterministic single-goal prediction
 58 is intrinsically mismatched to multimodal long-horizon exploration under partial observability, and
 59 why CFM offers a principled way to learn the resulting conditional anchor distribution. Extensive
 60 experiments, including within-dataset difficulty shift and cross-dataset transfer, demonstrate that the
 61 proposed framework achieves more robust generalization, higher reconstruction quality, and more
 62 efficient exploration in open environments.

63 2 Active 3D Mapping with Open-Environment Generalization

64 Let $\mathcal{S}$ denote an unknown scene, and let $\mathcal{X}^{\text{GT}} \subset \mathbb{R}^3$ denote its ground-truth surface. Active 3D
 65 mapping seeks to control an agent to acquire observations and reconstruct the scene as completely
 66 as possible under a finite budget T . At each time step t , the agent receives an RGB-D observation
 67 I_t and predicts the next pose $c_t = (c_t^{\text{pos}}, c_t^{\text{rot}})$ based on the history $\mathcal{H}_t = \{(I_0, c_0), \dots, (I_t, c_t)\}$.

This defines a partially observable sequential decision problem, where the high-level action satisfies $a_t \sim \pi(\cdot | \mathcal{H}_t)$. In this work, a_t is instantiated as a long-horizon exploration anchor, and executable candidate paths are recovered from sampled anchors using an explicit obstacle-aware planner.

Beyond within-distribution mapping performance, we focus on generalization to unseen scenes in open environments. Let $p_{\text{tr}}(\mathcal{S})$ and $p_{\text{te}}(\mathcal{S})$ denote the training and test scene distributions, where $p_{\text{tr}}(\mathcal{S}) \neq p_{\text{te}}(\mathcal{S})$ is allowed. Given the reconstructed point cloud $\mathcal{P}_t$, we measure mapping quality by the surface coverage $\text{Cov}(\mathcal{P}_t, \mathcal{X}^{\text{GT}})$, and use $\text{AUC}(\pi_\theta) = \frac{1}{T} \sum_{t=1}^T \text{Cov}(\mathcal{P}_t, \mathcal{X}^{\text{GT}})$ to quantify intermediate mapping efficiency. The overall objective is

$$\max_{\theta} \mathbb{E}_{\mathcal{S} \sim p_{\text{te}}(\mathcal{S})} [\lambda_1 \text{Cov}(\mathcal{P}_T, \mathcal{X}^{\text{GT}}) + \lambda_2 \text{AUC}(\pi_\theta)]. \quad (1)$$

3 Method

3.1 Overview

Active mapping in open environments exhibits two key challenges. First, the same local observation may admit multiple plausible long-horizon exploration directions under partial observability. Second, after geometric planning, these hypotheses often produce many similar candidates, which increases redundancy in downstream decision-making. To address this, we decompose long-horizon exploration into a hierarchical process. We first represent the current mapping progress by a 2D projected state. Specifically, let $\mathcal{P}_t$ denote the accumulated reconstructed point cloud at time t . We partition $\mathcal{P}_t$ into K horizontal slices and project them into K local density maps, while the trajectory history is projected into a visitation map, yielding $E_t = \{I_{t,1}^{\text{pc}}, \dots, I_{t,K}^{\text{pc}}, I_t^{\text{hist}}\}$. The shared context feature is then obtained by $z_t = f_\phi(E_t)$, where f_ϕ is a mapping-progress encoder. Based on z_t , we predict a local obstacle map $\hat{O}_t = g_\psi(z_t)$, which is used for subsequent feasible path planning. Built on the context feature z_t , our method proceeds in four stages, as illustrated in Fig. 3:

$$E_t \xrightarrow{\text{Encoder}} z_t \xrightarrow{\text{CFM}} \mathcal{A}_t \xrightarrow{\text{Planning+Clustering}} \{\mathcal{M}_t^{(m)}\}_{m=1}^{M_t} \xrightarrow{\text{Selection}} \tau_t^*. \quad (2)$$

Conditional Flow Matching (CFM) first generates a multimodal anchor set $\mathcal{A}_t$ for long-horizon exploration. The anchors are then converted into candidate paths, clustered into exploration modes $\{\mathcal{M}_t^{(m)}\}_{m=1}^{M_t}$, and hierarchically selected to obtain the final path τ_t^* .

3.2 Conditional Flow Matching for Multimodal Anchors

The purpose of this subsection is two-fold. First, we show why deterministic single-goal prediction is intrinsically mismatched to multimodal long-horizon exploration under partial observability. Second, based on this observation, we show why learning a conditional distribution over coarse anchors naturally leads to a CFM-based formulation in a low-dimensional decision space.

Revisiting the learning target. In open-environment active 3D mapping, the same local observation may correspond to multiple plausible long-horizon exploration directions. Let $p(a | z_t)$ denote the conditional distribution of feasible long-horizon exploration anchors under the current context z_t . Suppose one instead learns a deterministic predictor $\hat{a}(z_t)$ under the standard squared loss:

$$\mathcal{R}(\hat{a}; z_t) = \mathbb{E}_{a \sim p(a|z_t)} [\|\hat{a}(z_t) - a\|_2^2]. \quad (3)$$

Proposition 3.1 (Deterministic single-goal collapse). *The Bayes-optimal deterministic predictor of Eq. 3 is the conditional mean*

$$\hat{a}^*(z_t) = \mathbb{E}[a | z_t]. \quad (4)$$

Moreover, when $p(a | z_t)$ is multimodal and its feasible support is nonconvex due to scene geometry, $\hat{a}^(z_t)$ may lie outside any high-density mode of $p(a | z_t)$ and may even become infeasible for downstream planning.*

Proof sketch. Eq. 3 is a standard conditional least-squares objective, whose minimizer is the conditional mean in Eq. 4. When the anchor distribution contains several separated feasible modes, however, this mean generally lies between them. Under obstacle-induced nonconvex geometry, such an averaged anchor can fall into a blocked or low-probability region, making it neither representative nor executable. A full proof is provided in Appendix B.1.1.

Proposition 3.1 formalizes the mismatch of single-goal prediction: under multimodal long-horizon futures, deterministic regression is mean-seeking rather than mode-preserving. Therefore, the correct learning target is not a single long-horizon goal, but the full conditional anchor distribution $p(a | z_t)$.

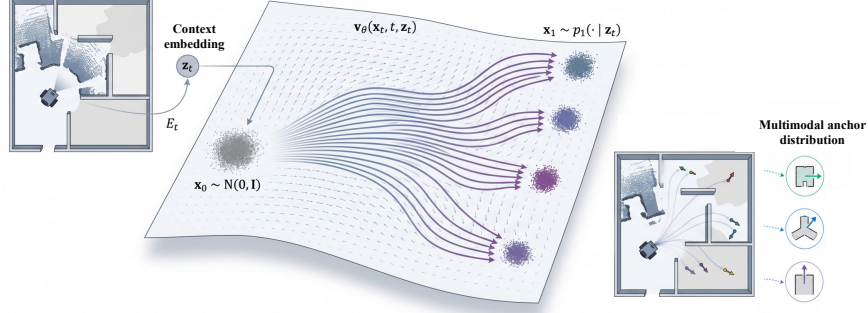


Figure 2: **Conditional Flow Matching for multimodal long-horizon anchors.** CFM transports samples from a simple prior $x_0 \sim \mathcal{N}(0, I)$ toward the target anchor distribution $x_1 \sim p_1(\cdot | z_t)$, yielding multiple plausible long-horizon exploration hypotheses under the same partial observation. Unlike directly generating full trajectories, this formulation models coarse exploration intent in a compact space and remains naturally compatible with downstream planning.

A low-dimensional conditional anchor space. A way to preserve multimodality would be to generate full trajectories. However, the role of the long-horizon proposal is to express *where to explore next* at a coarse semantic level, while detailed geometric feasibility is naturally handled by the explicit planner using the obstacle map $\hat{O}_t$. This suggests separating high-level long-horizon intent from low-level path realization, and introducing a compact variable for decision making.

We therefore define the long-horizon exploration anchor as

$$a = (a_{xy}, a_\theta) \in \mathbb{R}^4, \quad a_{xy} = (x_a, y_a), \quad a_\theta = (\sin \theta_a, \cos \theta_a), \quad (5)$$

where a_{xy} and a_θ represent the position and orientation of the long-horizon goal, respectively. This low-dimensional parameterization isolates long-horizon exploration intent from detailed path geometry, leaving fine-grained execution to the downstream planner.

Conditional transport via flow matching. Having established that the desired learning target is the conditional anchor distribution $p(a | z_t)$, we next need a principled way to model it. Conditional Flow Matching (CFM) is particularly suitable here, because it directly learns a conditional transport from a simple prior to the target distribution in continuous space, as illustrated in Fig. 2.

During training, we set the source distribution as a standard Gaussian and the target samples as oracle anchors:

$$x_0 \sim p_0 = \mathcal{N}(0, I), \quad x_1 \sim p_1(\cdot | z_t), \quad (6)$$

where x_1 is extracted from anchors in a high-reward path bank. Using the linear interpolation path

$$x_t = (1 - t)x_0 + tx_1, \quad t \sim \mathcal{U}(0, 1), \quad (7)$$

the target velocity is

$$u_t = x_1 - x_0. \quad (8)$$

We then train a conditional velocity field $v_\theta(x_t, t, z_t)$ by minimizing

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t, z_t) - u_t\|_2^2]. \quad (9)$$

The key point is that Eq. 9 is not merely a heuristic diversity objective; it is a conditional regression problem whose population optimum recovers the transport field of the anchor distribution.

Proposition 3.2 (CFM learns the conditional anchor transport field). *Let $p_t(\cdot | z_t)$ be the path induced by Eq. 7 between p_0 and $p_1(\cdot | z_t)$. Then the population minimizer of Eq. 9 is*

$$v^*(x_t, t, z_t) = \mathbb{E}[u_t | x_t, t, z_t]. \quad (10)$$

Thus, CFM learns the conditional transport field induced by the entire anchor distribution $p_1(\cdot | z_t)$, rather than regressing a single deterministic long-horizon target.

Proof sketch. Eq. 9 is a standard conditional least-squares objective. By the orthogonal decomposition of regression risk,

$$\mathbb{E}[\|v - u_t\|_2^2] = \mathbb{E}[\|v - \mathbb{E}[u_t | x_t, t, z_t]\|_2^2] + \mathbb{E}[\|u_t - \mathbb{E}[u_t | x_t, t, z_t]\|_2^2].$$

The second term is independent of v , so the minimum is achieved by Eq. 10. A full proof is provided in Appendix B.1.2.

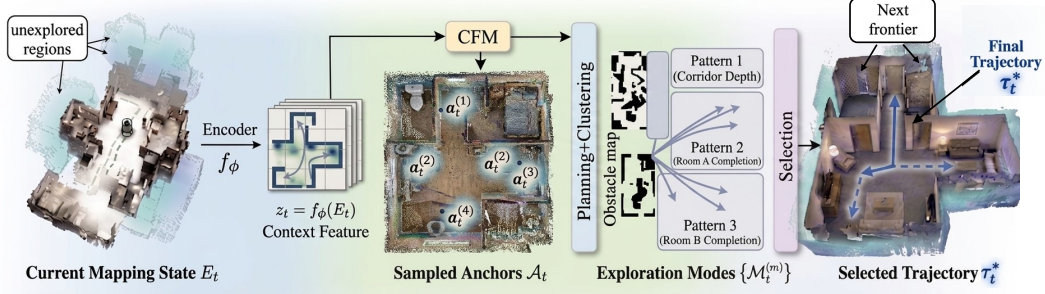


Figure 3: **Overview of our method.** Given the current mapping state E_t , a mapping-progress encoder produces the context feature z_t . Conditional Flow Matching (CFM) then generates a multimodal set of long-horizon exploration anchors $\mathcal{A}_t$, which are converted into executable candidate paths and clustered into exploration modes $\{\mathcal{M}_t^{(m)}\}_{m=1}^{M_t}$. Finally, a hierarchical selection module first selects the most promising mode and then reranks paths within that mode to output the final trajectory τ_t^* . The overall training objective is presented in Appendix B.2.

Proposition 3.2 shows why CFM is a principled choice for our setting: once the learning target is reformulated from a deterministic goal to a conditional anchor distribution, CFM directly learns its conditional transport in the low-dimensional anchor space. Compared with directly generating full trajectories, this formulation is easier to optimize, preserves multimodal long-horizon hypotheses, and remains naturally compatible with explicit planning.

During inference, we sample K anchors from the learned conditional prior:

$$\mathcal{A}_t = \{a_t^{(1)}, \dots, a_t^{(K)}\}, \quad a_t^{(k)} \sim p_\theta(a \mid z_t). \quad (11)$$

These samples constitute the multimodal long-horizon exploration hypotheses at the current time step. To encourage coverage over distinct long-horizon exploration modes, we further introduce a diversity regularization term:

$$\mathcal{L}_{\text{div}} = \frac{1}{K(K-1)} \sum_{i \neq j} \exp \left(-\frac{\|a_t^{(i)} - a_t^{(j)}\|_2^2}{\sigma^2} \right). \quad (12)$$

Minimizing this term encourages different anchors to remain sufficiently dispersed, thereby covering diverse long-horizon exploration modes. In this way, the proposed CFM module provides a set of diverse, context-consistent exploration hypotheses for downstream planning and selection.

3.3 Planning and Anchor-Guided Mode Construction.

The previous subsection motivates modeling long-horizon exploration as a conditional anchor distribution rather than a single deterministic goal. However, anchors are only coarse intents and cannot be executed directly, and different anchors may still produce highly similar paths after planning. We therefore recover anchors into executable candidate paths via obstacle-aware planning, and further cluster them into representative exploration modes to reduce redundancy before hierarchical selection.

From Anchors to Executable Paths. Given the current pose c_t , anchor $a_t^{(k)}$, and obstacle map $\hat{O}_t$, we obtain the candidate path via an explicit planner:

$$\tau_t^{(k)} = \text{Plan}(c_t, a_t^{(k)}, \hat{O}_t), \quad (13)$$

where *Plan* is implemented as an A*-based [16] shortest path planner. To characterize each candidate path for downstream reasoning, we further extract a compact path descriptor

$$\varphi(\tau_t^{(k)}) = [g_k, r_k, c_k, n_k, h_k, u_k], \quad (14)$$

which respectively characterize the gain proxy, risk, cost, novelty, continuity deviation, and uncertainty statistics. They serve as structured intermediate features for the learned hierarchical selector in Sec. 3.4. Their exact computation is deferred to Appendix B.4.1.

From Candidate Paths to Exploration Modes. If selection is performed directly path-by-path on $\{\tau_t^{(k)}\}_{k=1}^K$, redundant candidates will interfere with decision-making. To circumvent this, we first discover more abstract exploration modes over the path set. Denoting the candidate path set as $\mathcal{T}_t = \{\tau_t^{(1)}, \dots, \tau_t^{(K)}\}$, we define the distance between any two paths τ_i, τ_j in the path space as:

$$d(\tau_i, \tau_j) = \lambda_1 d_{\text{end}}(\tau_i, \tau_j) + \lambda_2 d_{\text{shape}}(\tau_i, \tau_j) + \lambda_3 d_{\text{prefix}}(\tau_i, \tau_j) + \lambda_4 d_{\text{topo}}(\tau_i, \tau_j), \quad (15)$$

where d_{end} measures the endpoint discrepancy, d_{shape} measures the geometric shape discrepancy of the paths after resampling, d_{prefix} measures the prefix trajectory discrepancy to characterize short-term execution behaviors, and d_{topo} measures the topological difference between the two paths in terms of obstacle circumvention. The definitions of each component can be found in Appendix B.4.2.

Under the distance metric, we cluster the candidate paths into M_t exploration modes:

$$\mathcal{T}_t \longrightarrow \{\mathcal{M}_t^{(1)}, \dots, \mathcal{M}_t^{(M_t)}\}. \quad (16)$$

Each mode $\mathcal{M}_t^{(m)}$ corresponds to a class of long-horizon exploration hypotheses, such as “prioritize entering the left room”, “continue deep along the main corridor”, or “complete the current local region first before expanding farther”. Furthermore, we define a representative path $\tilde{\tau}_t^{(m)} \in \mathcal{M}_t^{(m)}$ for each mode, which is implemented as the medoid within that mode.

Simultaneously, we define mode statistics:

$$\chi_t^{(m)} = [|\mathcal{M}_t^{(m)}|, \bar{g}_m, \bar{r}_m, \text{Var}(g)_m, \text{Var}(r)_m, \rho_m], \quad (17)$$

where $|\mathcal{M}_t^{(m)}|$ denotes the mode size, $\bar{g}_m$ and $\bar{r}_m$ denote the average gain and average risk within the mode, $\text{Var}(g)_m$ and $\text{Var}(r)_m$ measure the dispersion of gain and risk, and ρ_m denotes the mode radius. Together with the representative path, these statistics provide a compact summary of the mode’s quality and internal consistency. As with the path descriptors above, they are used as auxiliary structured inputs to the downstream learned scorer, rather than as standalone selection criteria. Their exact definitions are provided in Appendix B.4.3.

Therefore, this module elevates multimodal anchor hypotheses into executable, comparable, and hierarchically structured exploration mode representations.

3.4 Hierarchical Mode Selection

The previous subsection organizes multimodal anchors into a set of exploration modes, but mode construction alone does not resolve the final decision. In open-environment active 3D mapping, the agent’s goal is to make robust long-horizon exploration decisions under partial observability and scene shift, rather than to score many redundant paths independently. If selection is still performed directly over all candidate paths, geometrically similar trajectories can interfere with scoring and bias the policy toward unstable local choices, which weakens generalization and reduces exploration efficiency. We therefore adopt a hierarchical selection strategy: first selecting a promising mode at the strategy level, and then reranking paths within that mode to obtain the final executable trajectory.

Mode-Level Scoring. For each mode, we construct a mode representation:

$$z_t^{(m)} = \Gamma(z_t, \tilde{\tau}_t^{(m)}, \chi_t^{(m)}), \quad (18)$$

where z_t is the shared context output by the context encoder, and $\Gamma(\cdot)$ represents the mode encoder used to fuse the shared context, representative path, and mode statistics. Its specific implementation is provided in Appendix B.5.1.

In open environments, a mode worth executing should not only possess a high expected gain but also simultaneously satisfy the criteria of controllable risk, low uncertainty, and continuity with existing execution plans, while preserving a degree of exploration diversity when necessary. To this end, we predict multi-head scores via a mode scorer:

$$(G_m, R_m, U_m, C_m, D_m) = s_\eta(z_t^{(m)}), \quad (19)$$

where G_m , R_m , U_m , C_m , and D_m represent the expected gain, execution risk, uncertainty, mode switching or cost, and exploration diversity reward of the mode, respectively. Here, $s_\eta(\cdot)$ represents a learnable mode-level scoring network, whose multi-head output definitions and supervision targets are detailed in Appendix B.5.2. Accordingly, the overall mode-level score is defined as:

$$S_m^{\text{mode}} = w_G G_m - w_R R_m - w_U U_m - w_C C_m + w_D D_m, \quad (20)$$

where $w_G, w_R, w_U, w_C, w_D \geq 0$ are learnable weights. Thus, the optimal mode selection at the current time step is:

$$m_t^* = \arg \max_m S_m^{\text{mode}}. \quad (21)$$

Intra-Mode Local Reranking. Within the selected mode $\mathcal{M}_t^{(m_t^*)}$, for any candidate path τ , we construct a path-level representation:

$$\zeta_t(\tau) = \Lambda(z_t, \tau, \varphi(\tau), \chi_t^{(m_t^*)}), \quad (22)$$

where $\Lambda(\cdot)$ represents the feature fusion function of the local reranker. Subsequently, the local reranker outputs four path-level scoring heads $\hat{G}(\tau)$, $\hat{R}(\tau)$, $\hat{C}(\tau)$, $\hat{H}(\tau)$, which respectively characterize path gain, path risk, path cost, and continuity penalty. Their specific implementations and supervision targets are provided in Appendix B.5.3. Consequently, the overall local path score is defined as:

$$S^{\text{local}}(\tau) = \hat{G}(\tau) - \lambda_R \hat{R}(\tau) - \lambda_C \hat{C}(\tau) - \lambda_H \hat{H}(\tau). \quad (23)$$

The final execution path is thus given by:

$$\tau_t^* = \arg \max_{\tau \in \mathcal{M}_t^{(m_t^*)}} S^{\text{local}}(\tau). \quad (24)$$

Thus, this section decouples long-range strategy reasoning from local path execution via hierarchical selection, improving stability and robustness in open-environment exploration.

4 Experiments

4.1 Experimental Setup

We evaluate our method on Matterport3D (MP3D) [17] and AiMDoom [8]. For AiMDoom, we adopt a 70/30 train/test split within each difficulty level. On AiMDoom, we report two standard active-mapping metrics: *Final Coverage*, which measures the final scene coverage at the end of an episode, and *AUC*, which evaluates reconstruction efficiency by the area under the coverage curve over time. Surface coverage is computed against the ground-truth mesh following [8]. We evaluate five trajectories per scene using identical random initial poses for different methods, and report the mean and standard deviation over all testing trajectories. For MP3D, we additionally report the completion metrics used in prior work, namely *Comp. (%)* and *Comp. (cm)*.

We follow the same mapping-state construction and basic training protocol as [8] for fair comparison. Our model takes projected local map observations and trajectory history as input, and predicts an obstacle map for downstream proposal generation and planning. *Additional dataset configurations, metric definitions, and full implementation details are provided in Appendix B.3.*

4.2 Comparison with the State of the Art

AiMDoom dataset. Table 1 shows that our method achieves the best performance on all four AiMDoom difficulty levels. Compared with NBP [8], it consistently improves both Final Coverage and AUC, with larger gains on the harder splits. In particular, Final Coverage improves from 0.734 to 0.781 on Normal, from 0.618 to 0.662 on Hard, and from 0.472 to 0.534 on Insane, while AUC also improves consistently. This trend is particularly meaningful because AiMDoom is designed to

Table 1: **Evaluation results on AiMDoom Dataset.** For each difficulty level, all baseline models, including ours, are trained from scratch on the corresponding training set to ensure a fair comparison.

	Simple		Normal		Hard		Insane	
	Final Cov.	AUCs	Final Cov.	AUCs	Final Cov.	AUCs	Final Cov.	AUCs
Random	0.323±0.156	0.270±0.135	0.190±0.124	0.152±0.103	0.124±0.082	0.088±0.060	0.074±0.048	0.050±0.035
FBE [18]	0.760±0.174	0.605±0.171	0.565±0.139	0.415±0.109	0.425±0.114	0.311±0.080	0.330±0.097	0.239±0.079
SCONE [19]	0.577±0.173	0.483±0.138	0.412±0.114	0.313±0.087	0.290±0.093	0.210±0.072	0.196±0.079	0.140±0.060
MACARONS [7]	0.599±0.200	0.479±0.172	0.418±0.120	0.314±0.088	0.302±0.097	0.218±0.070	0.192±0.078	0.139±0.058
NBP [8]	0.879±0.142	0.692±0.156	0.734±0.142	0.526±0.112	0.618±0.153	0.432±0.115	0.472±0.095	0.312±0.073
Ours	0.913±0.138	0.707±0.141	0.781±0.125	0.554±0.096	0.662±0.139	0.446±0.112	0.534±0.073	0.325±0.089

Table 2: **Comparison on the MP3D dataset.**

Method	Comp. (%) ↑	Comp. (cm) ↓
Random	45.67	26.53
FBE [18]	71.18	9.78
UPEN [3]	69.06	10.60
OccAnt [6]	71.72	9.40
ANM [4]	73.15	9.11
NBP [8]	79.38	6.78
Ours	82.67	5.67

Table 3: **Train on AiMDoom test on MP3D.**

Method	Comp. (%) ↑	Comp. (cm) ↓
Random	32.12	29.36
FBE [18]	56.73	14.45
UPEN [3]	54.70	13.09
OccAnt [6]	58.49	14.18
ANM [4]	60.22	11.78
NBP [8]	61.81	10.22
Ours	68.45	9.11

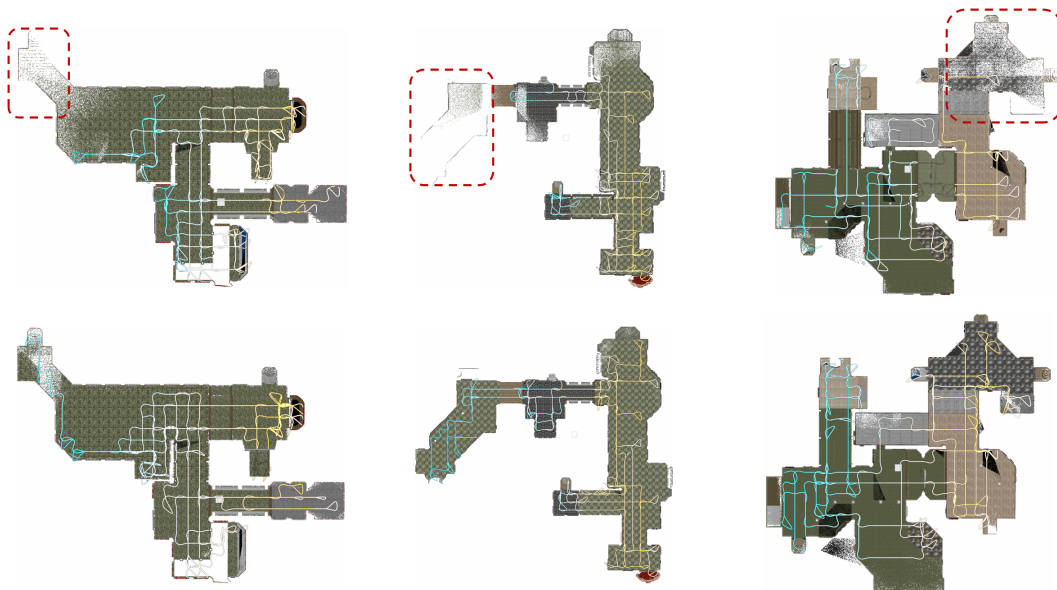


Figure 4: **Qualitative comparison between the SOTA method NBP (Top) and Ours (Bottom).** For each scene, both methods start from the same initial pose. The dashed boxes highlight representative regions that remain unexplored by NBP but are successfully recovered by our method. This difference is not merely local: NBP tends to commit early to a single dominant long-horizon branch, which induces a narrower global coverage pattern and leaves weakly indicated but still informative continuations unexplored. In contrast, our method preserves multiple plausible long-horizon exploration hypotheses before downstream planning and selection, enabling more balanced expansion across alternative scene continuations and ultimately yielding more complete scene coverage.

increase scene ambiguity and long-horizon planning difficulty as the level becomes harder. The results therefore suggest that preserving multiple plausible long-horizon hypotheses makes exploration more robust under increasing uncertainty, rather than committing prematurely to a single predicted branch.

MP3D dataset. As shown in Table 2, our method achieves the best results on MP3D, reaching 82.67 in Comp. (%) and 5.67 in Comp. (cm). Compared with NBP [8], this corresponds to a gain of 3.29 points in completion ratio and a reduction of 1.11 cm in completion error. The improvement in both metrics is important: the higher completion ratio indicates better scene coverage, while the lower completion error shows that the reconstructed geometry is also more accurate. These results suggest that the multimodal long-horizon modeling is not only effective on synthetic open-environment benchmarks, but also transfers well to realistic indoor scenes with more complex layouts.

4.3 Generalization under Out-of-Distribution Settings

We evaluate OOD generalization from two perspectives: *within-dataset difficulty shift* on AiMDoom and *cross-dataset transfer* from AiMDoom to MP3D. The results consistently support our central claim: preserving multiple long-horizon exploration hypotheses improves generalization, whereas single-goal methods generalize poorly under scene shift.

Cross-dataset transfer from AiMDoom to MP3D. Table 3 reports zero-shot transfer from AiMDoom to MP3D. Despite the large domain gap, our method achieves the best performance, reaching 68.45 in Comp. (%) and 9.11 in Comp. (cm), improving over NBP by 6.64 points in completion ratio and 1.11 cm in completion error. This result suggests that multimodal long-horizon modeling transfers better across unseen scene distributions than single-goal prediction.

Within-dataset difficulty shift on AiMDoom. Table 4 reports two OOD protocols on AiMDoom. Under both settings, our method consistently outperforms NBP, with especially clear gains on the harder splits. This suggests that preserving multiple plausible long-horizon exploration paths is more robust to increasing scene ambiguity than single-goal prediction, which tends to commit early to one branch and thus generalizes poorly under difficulty shift.

To further strengthen the OOD evaluation, Appendix C.1 reports additional cross-domain generalization results on *GLEAM-Bench* [9], while Appendix C.2 provides controlled analyses showing that the weak generalization of prior methods is closely tied to deterministic single-goal prediction rather than merely insufficient candidate enumeration.

Table 4: **Generalization under difficulty shift on AiMDoom.** We report two complementary protocols. Protocol A evaluates cross-difficulty transfer by training on the *Simple* split only and directly testing on all difficulty levels. Protocol B evaluates leave-one-difficulty-out generalization by training on *Simple + Normal + Hard* and directly testing on the held-out *Insane* split.

Protocol	Method	Simple		Normal		Hard		Insane	
		Final Cov.	AUC	Final Cov.	AUC	Final Cov.	AUC	Final Cov.	AUC
Protocol A: Train on Simple, test on all difficulty levels									
	Random	0.323±0.156	0.270±0.135	0.174±0.112	0.132±0.091	0.101±0.075	0.072±0.054	0.062±0.041	0.041±0.030
	FBE [18]	0.760±0.174	0.605±0.171	0.512±0.155	0.364±0.124	0.368±0.132	0.255±0.098	0.284±0.110	0.188±0.088
	SCONE [19]	0.577±0.173	0.483±0.138	0.354±0.128	0.245±0.104	0.213±0.111	0.142±0.082	0.145±0.092	0.094±0.071
	MACARONS [7]	0.599±0.200	0.479±0.172	0.368±0.141	0.252±0.115	0.225±0.124	0.148±0.087	0.151±0.096	0.098±0.075
	NBP [8]	0.879±0.142	0.692±0.156	0.695±0.158	0.502±0.128	0.562±0.161	0.401±0.122	0.434±0.114	0.275±0.098
	Ours	0.913±0.138	0.707±0.141	0.727±0.152	0.526±0.133	0.597±0.174	0.403±0.121	0.467±0.106	0.313±0.101
Protocol B: Train on Simple + Normal + Hard, test on held-out Insane									
	Random	–	–	–	–	–	–	0.068±0.045	0.048±0.038
	FBE [18]	–	–	–	–	–	–	0.312±0.124	0.201±0.105
	SCONE [19]	–	–	–	–	–	–	0.168±0.101	0.112±0.082
	MACARONS [7]	–	–	–	–	–	–	0.182±0.115	0.115±0.088
	NBP [8]	–	–	–	–	–	–	0.421±0.128	0.302±0.112
	Ours	–	–	–	–	–	–	0.438±0.119	0.317±0.089

4.4 Ablation Study

To verify the generalization of our method to unseen open-world scenes, all experiments in this subsection are conducted under a cross-dataset setting: *trained on AiMDoom and tested on MP3D*.

Component Analysis. Table 5 reports a progressive validation of the proposed pipeline, where MAG, EMC, and HMS denote multimodal anchor generation, exploration-mode clustering, and hierarchical mode/path selection, respectively. As modules are introduced stage by stage, the performance improves consistently under the cross-dataset OOD setting. The largest gain comes from adding MAG, which improves Comp. (%) from 61.81 to 66.21 and reduces Comp. (cm) from 10.22 to 9.64. This is consistent with our central claim that the main limitation of existing methods lies in deterministic single-goal prediction: replacing it with multimodal anchor generation already yields a much stronger long-horizon decision mechanism. Adding EMC further improves the results to 67.60 and 9.36, showing that organizing raw candidate paths into exploration modes is beneficial for reducing redundancy among geometrically similar proposals. Finally, adding HMS yields the best performance of 68.45 and 9.11, suggesting that hierarchical mode/path selection further stabilizes downstream decision making by decoupling high-level strategy choice from local path reranking.

We have provided more detailed supplementary ablation analyses in the appendix. In particular, Appendix C.2 compares *different proposal-generation paradigms* to examine whether the weak generalization of prior methods is fundamentally tied to deterministic single-goal prediction, and further studies *post-hoc multi-goal extensions* to test whether simply enumerating more candidate goals at inference time can replace explicit multimodal modeling. In addition, Appendix C.3 reports sensitivity studies of the number of sampled anchors, CFM sampling steps, and clustering granularity.

Table 5: **Progressive validation of the proposed pipeline under the cross-dataset OOD setting.**

Method				OOD Setting	
Baseline	MAG	EMC	HMS	Comp. (%) ↑	Comp. (cm) ↓
✓				61.81	10.22
✓	✓			66.21	9.64
✓	✓	✓		67.60	9.36
✓	✓	✓	✓	68.45	9.11

5 Conclusion

In this paper, we study active 3D mapping in open environments, where partial observability and scene shift make deterministic single-goal prediction difficult to generalize. To address this issue, we reformulate long-horizon decision making as a conditional multimodal proposal generation problem and propose a flow-matching-based framework for multimodal anchor modeling. The generated anchors are further converted into executable candidate paths, compressed into exploration modes, and selected through a hierarchical decision process. Extensive experiments show that our method consistently improves reconstruction quality, exploration efficiency, and robustness under both difficulty shift and cross-dataset transfer. These results suggest that preserving multimodal long-horizon exploration hypotheses is a promising direction for open-environment active mapping.

References

- [1] Joseph Ortiz, Alexander Clegg, Jing Dong, Edgar Sucar, David Novotny, Michael Zollhoefer, and Mustafa Mukadam. isdf: Real-time neural signed distance fields for robot perception. *arXiv preprint arXiv:2204.02296*, 2022.
- [2] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J Davison. imap: Implicit mapping and positioning in real-time. In *ICCV*, pages 6229–6238, 2021.
- [3] Georgios Georgakis, Bernadette Bucher, Anton Arapin, Karl Schmeckpeper, Nikolai Matni, and Kostas Daniilidis. Uncertainty-driven planner for exploration and navigation. In *ICRA*, pages 11295–11302, 2022.
- [4] Zike Yan, Haoxiang Yang, and Hongbin Zha. Active neural mapping. In *ICCV*, pages 10981–10992, 2023.
- [5] Daryl Peralta, Joel Casimiro, Aldrin Michael Nilles, Justine Aletta Aguilar, Rowel Atienza, and Rhandley Cajote. Next-best view policy for 3d reconstruction. In *ECCV Workshops*, pages 558–573, 2020.
- [6] Santhosh K Ramakrishnan, Ziad Al-Halah, and Kristen Grauman. Occupancy anticipation for efficient exploration and navigation. In *ECCV*, pages 400–418, 2020.
- [7] Antoine Guédon, Tom Monnier, Pascal Monasse, and Vincent Lepetit. Macarons: Mapping and coverage anticipation with rgb online self-supervision. In *CVPR*, pages 940–951, 2023.
- [8] Shiyao Li, Antoine Guedon, Clémentin Boittiaux, Shizhe Chen, and Vincent Lepetit. Nextbestpath: Efficient 3d mapping of unseen environments. In *ICLR*, 2025.
- [9] Xiao Chen, Tai Wang, Quanyi Li, Tao Huang, Jiangmiao Pang, and Tianfan Xue. Gleam: Learning generalizable exploration policy for active mapping in complex 3d indoor scene. In *ICCV*, pages 5558–5568, 2025.
- [10] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [12] Jeongyong Yang, Seunghwan Jang, and SooJean Han. Safeflowmatcher: Safe and fast planning using flow matching with control barrier functions, 2026. URL <https://arxiv.org/abs/2509.24243>.
- [13] Samiran Gode, Abhijeet Nayak, Débora N.P. Oliveira, Michael Krawez, Cordelia Schmid, and Wolfram Burgard. Flownav: Combining flow matching and depth priors for efficient navigation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 17762–17768, 2025. doi: 10.1109/IROS60139.2025.11246634.
- [14] Sen Wang, Le Wang, Sanping Zhou, Jingyi Tian, Jiayi Li, Haowen Sun, and Wei Tang. Flowram: Grounding flow matching policy with region-aware mamba framework for robotic manipulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12176–12186, 2025.
- [15] Zebin Xing, Xingyu Zhang, Yang Hu, Bo Jiang, Tong He, Qian Zhang, Xiaoxiao Long, and Wei Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *CVPR*, pages 1602–1611, 2025.
- [16] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [17] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from rgb-d data in indoor environments. *arXiv preprint arXiv:1709.06158*, 2017.
- [18] Brian Yamauchi. A frontier-based approach for autonomous exploration. In *CIRA*, pages 146–151, 1997.
- [19] Antoine Guédon, Pascal Monasse, and Vincent Lepetit. Scone: Surface coverage optimization in unknown environments by volumetric integration. In *NeurIPS*, 2022.
- [20] George Stoica, Sayak Paul, Matthew Wallingford, Vivek Ramanujan, Abhay Nori, Winson Han, Ali Farhadi, Ranjay Krishna, and Judy Hoffman. Posterior augmented flow matching, 2026. URL <https://arxiv.org/abs/2605.00825>.
- [21] Andrew J Davison and David W Murray. Simultaneous localization and map-building using active vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7):865–880, 2002.

- 346 [22] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid,
347 and John J Leonard. Past, present, and future of simultaneous localization and mapping: Toward the
348 robust-perception age. *IEEE Transactions on robotics*, 32(6):1309–1332, 2016.
- 349 [23] Iker Lluvia, Elena Lazkano, and Ander Ansuategi. Active mapping and robot exploration: A survey.
350 *Sensors*, 21(7):2445, 2021.
- 351 [24] Julio A Placed, Jared Strader, Henry Carrillo, Nikolay Atanasov, Vadim Indelman, Luca Carlone, and
352 José A Castellanos. A survey on active simultaneous localization and mapping: State of the art and new
353 frontiers. *arXiv preprint arXiv:2207.00254*, 2022.
- 354 [25] Tao Chen, Saurabh Gupta, and Abhinav Gupta. Learning exploration policies for navigation. In *International Conference on Learning Representations*, 2019.
- 356 [26] Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov.
357 Learning to explore using active neural slam. In *International Conference on Learning Representations*,
358 2020.
- 359 [27] Devendra Singh Chaplot, Dhiraj Gandhi, Saurabh Gupta, Abhinav Gupta, and Ruslan Salakhutdinov.
360 Learning to explore using active neural slam. *arXiv preprint arXiv:2004.05155*, 2020.

361	Appendix	
362	A Related Work	12
363	B Supplementary Method and Theory	13
364	B.1 Additional Theoretical Analysis for CFM-Based Multimodal Anchors	13
365	B.2 Training and Inference Details	15
366	B.3 Network and Implementation Details	18
367	B.4 Detailed Definitions of Path Features and Distance Metrics	20
368	B.5 Supervision Targets for Mode Scoring and Local Reranking	24
369	C Supplementary Experiments	28
370	C.1 Extended Generalization Evaluation on GLEAM-Bench [9]	28
371	C.2 Why Deterministic Single-Goal Prediction Limits Generalization	28
372	C.3 Hyperparameter Settings and Sensitivity Analysis	30
373	C.4 Computational Cost	32
374	D Supplementary Visualizations and Failure Analysis	33
375	D.1 Additional Qualitative Comparison with NBP	33
376	D.2 Failure Cases and Limitations	33

377 A Related Work

378 **From Passive Reconstruction to Active 3D Mapping.** Traditional 3D reconstruction methods
379 mainly study how to recover scene geometry from a fixed set of observations, without explicitly
380 considering how future observations should be acquired [1, 2]. In contrast, active 3D mapping [3, 4]
381 requires an embodied agent to actively select future viewpoints or trajectories while reconstructing
382 the scene, making exploration planning a core part of the problem. Early active mapping methods are
383 largely heuristic-based, including frontier-based exploration and related next-best-view strategies [18,
384 5], while later learning-based methods predict local viewpoints or navigation goals from the current
385 mapping state [6, 7]. Although these methods improve exploration efficiency, they are still mostly
386 studied in relatively simple or closed-set environments, where the training and test scenes are highly
387 correlated and the scene distributions are not sufficiently diverse. As a result, their generalization to
388 unseen open environments remains limited.

389 **Open-Environment Active Mapping and Generalization.** Recent benchmarks such as AiM-
390 Doom [8] and GLEAM-Bench [9] extend active 3D mapping to more diverse indoor scenes and more
391 challenging cross-scene settings, making open-environment generalization a central issue. Corre-
392 spondingly, recent methods begin to consider long-horizon decision making rather than purely local
393 viewpoint selection. For example, NBP [8] predicts a long-term goal together with an obstacle map to
394 enable path-level planning. However, existing methods on open-environment benchmarks still largely
395 formulate long-horizon exploration as single-point prediction. This formulation is fundamentally
396 limited under partial observability, where the same local observation may correspond to multiple
397 plausible future exploration directions. In this work, we focus on this limitation and study how to
398 make the long-horizon decision model itself more robust and transferable in unseen environments.

399 **Generative Modeling for Multimodal Long-Horizon Decisions.** Generative models have recently
400 shown strong potential for decision-making problems with multimodal futures, especially in embodied
401 decision-making and motion-planning tasks [11–15]. Compared with deterministic regression,
402 they can represent multiple plausible decisions conditioned on the current context. Among them,
403 Conditional Flow Matching (CFM) [10] learns a continuous transport from a simple prior to a
404 conditional target distribution and provides a stable and efficient framework for multimodal generation.
405 Recent studies in flow matching, such as PAFM [20], further suggest that preserving multiple plausible
406 continuations can be important for robust learning. This observation is conceptually aligned with
407 our motivation. In contrast to existing flow-based embodied decision-making methods that mainly
408 generate full trajectories or action sequences, we use CFM to model a conditional distribution over
409 coarse long-horizon exploration anchors for active 3D mapping, preserving multimodal exploration
410 hypotheses while remaining compatible with explicit planning.

411 **Active 3D Mapping vs. Active SLAM.** Active 3D mapping is closely related to active SLAM [21–
412 26], since both require an embodied agent to actively move, acquire observations, and update a

map. However, the two problems emphasize different objectives. Active SLAM jointly considers localization, mapping, and planning, and often focuses on reducing uncertainty in robot pose or map estimation to support reliable navigation. In contrast, active 3D mapping focuses more directly on exhaustive geometric reconstruction, aiming to select future viewpoints or paths that improve scene coverage and reconstruction quality, typically assuming the agent pose is available or can be reliably estimated. This makes active 3D mapping relevant to applications such as autonomous exploration, robotic inspection, and digital-twin construction, where an agent should actively gather informative observations to improve scene reconstruction while moving through an unknown environment. Compared with active SLAM, it places more emphasis on reconstruction quality and scene coverage.

B Supplementary Method and Theory

This section provides supplementary theoretical analysis, complete proofs, and detailed methodological descriptions of the proposed framework.

B.1 Additional Theoretical Analysis for CFM-Based Multimodal Anchors

In this appendix, we provide full proofs for Proposition 3.1 and Proposition 3.2 in Sec. 3.2.

B.1.1 Proof of Proposition 3.1

We restate the proposition for completeness.

Proposition B.1. *Let $p(a \mid z_t)$ denote the conditional distribution of feasible long-horizon exploration anchors under context z_t , and consider deterministic prediction under the squared loss*

$$\mathcal{R}(\hat{a}; z_t) = \mathbb{E}_{a \sim p(a \mid z_t)} [\|\hat{a}(z_t) - a\|_2^2].$$

Then the Bayes-optimal deterministic predictor is

$$\hat{a}^*(z_t) = \mathbb{E}[a \mid z_t].$$

Moreover, when $p(a \mid z_t)$ is multimodal and its feasible support is nonconvex due to scene geometry, $\hat{a}^(z_t)$ may lie outside any high-density mode of $p(a \mid z_t)$ and may even become infeasible for downstream planning.*

Proof. For a fixed context z_t , denote $h = \hat{a}(z_t)$. Then

$$\mathcal{R}(h; z_t) = \mathbb{E}_{a \sim p(a \mid z_t)} [\|h - a\|_2^2].$$

Let $\mu_t = \mathbb{E}[a \mid z_t]$ be the conditional mean. Expanding around μ_t gives

$$\mathcal{R}(h; z_t) = \mathbb{E} [\|h - \mu_t + \mu_t - a\|_2^2 \mid z_t] \tag{25}$$

$$= \mathbb{E} [\|h - \mu_t\|_2^2 + 2(h - \mu_t)^\top (\mu_t - a) + \|\mu_t - a\|_2^2 \mid z_t]. \tag{26}$$

Since $h - \mu_t$ is deterministic conditioned on z_t and

$$\mathbb{E}[\mu_t - a \mid z_t] = \mu_t - \mathbb{E}[a \mid z_t] = 0,$$

the cross term vanishes, yielding

$$\mathcal{R}(h; z_t) = \|h - \mu_t\|_2^2 + \mathbb{E} [\|\mu_t - a\|_2^2 \mid z_t]. \tag{27}$$

The second term in Eq. 27 does not depend on h , so the unique minimizer is

$$h^* = \mu_t = \mathbb{E}[a \mid z_t].$$

Therefore,

$$\hat{a}^*(z_t) = \mathbb{E}[a \mid z_t].$$

We next show why this deterministic optimum can be problematic under multimodal and nonconvex feasible supports. Consider a simple two-mode anchor distribution under a fixed context:

$$p(a \mid z_t) = \frac{1}{2}\delta_{a^{(1)}} + \frac{1}{2}\delta_{a^{(2)}},$$

where $a^{(1)}$ and $a^{(2)}$ correspond to two separated feasible exploration branches, for example two distinct corridor continuations around an obstacle. Then

$$\hat{a}^*(z_t) = \frac{a^{(1)} + a^{(2)}}{2}.$$

439 If the feasible anchor set is nonconvex due to obstacles or scene topology, this midpoint need not be
 440 feasible. Even when it remains feasible, it can lie in a low-density region between the two modes and
 441 thus fail to represent either plausible long-horizon branch. Hence deterministic single-goal regression
 442 is intrinsically mean-seeking and can collapse multimodal futures into a brittle anchor. $\square$

443 B.1.2 Proof of Proposition 3.2

444 We restate the proposition for completeness.

Proposition B.2. *Let $p_t(\cdot \mid z_t)$ be the probability path induced by the linear interpolation*

$$x_t = (1 - t)x_0 + tx_1, \quad t \sim \mathcal{U}(0, 1),$$

between p_0 and $p_1(\cdot \mid z_t)$, where the target velocity is $u_t = x_1 - x_0$. Then the population minimizer of

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t, z_t) - u_t\|_2^2]$$

is

$$v^*(x_t, t, z_t) = \mathbb{E}[u_t \mid x_t, t, z_t].$$

Thus, CFM learns the conditional transport field induced by the full anchor distribution $p_1(\cdot \mid z_t)$.

445 *Proof.* For simplicity, define

$$X = (x_t, t, z_t), \quad U = u_t.$$

446 Then the population objective can be written as

$$\mathcal{L}(v) = \mathbb{E} [\|v(X) - U\|_2^2].$$

447 Applying the tower property,

$$\mathcal{L}(v) = \mathbb{E} [\mathbb{E} [\|v(X) - U\|_2^2 \mid X]].$$

448 Therefore, it suffices to minimize the inner conditional expectation pointwise for each fixed X . Let

$$m(X) = \mathbb{E}[U \mid X].$$

449 Then

$$v(X) - U = (v(X) - m(X)) + (m(X) - U),$$

450 and thus

$$\|v(X) - U\|_2^2 = \|v(X) - m(X)\|_2^2 + \|m(X) - U\|_2^2 \quad (28)$$

$$+ 2(v(X) - m(X))^\top (m(X) - U). \quad (29)$$

451 Taking conditional expectation given X yields

$$\mathbb{E} [\|v(X) - U\|_2^2 \mid X] = \|v(X) - m(X)\|_2^2 + \mathbb{E} [\|m(X) - U\|_2^2 \mid X] \quad (30)$$

$$+ 2(v(X) - m(X))^\top \mathbb{E}[m(X) - U \mid X]. \quad (31)$$

452 Since $m(X) = \mathbb{E}[U \mid X]$, we have

$$\mathbb{E}[m(X) - U \mid X] = m(X) - \mathbb{E}[U \mid X] = 0.$$

453 Hence the cross term vanishes and

$$\mathbb{E} [\|v(X) - U\|_2^2 \mid X] = \|v(X) - m(X)\|_2^2 + \mathbb{E} [\|m(X) - U\|_2^2 \mid X]. \quad (32)$$

454 The second term in Eq. 32 does not depend on v , so the unique minimizer is

$$v^*(X) = m(X) = \mathbb{E}[U \mid X].$$

455 Substituting back $X = (x_t, t, z_t)$ and $U = u_t$ gives

$$v^*(x_t, t, z_t) = \mathbb{E}[u_t \mid x_t, t, z_t].$$

456 This shows that the optimal CFM vector field is the conditional mean transport velocity along the
 457 probability path induced by the full anchor distribution $p_1(\cdot \mid z_t)$. Therefore, CFM does not regress a
 458 single deterministic target; instead, it learns the conditional transport field associated with the entire
 459 multimodal anchor distribution. $\square$

B.1.3 Discussion

Proposition 3.1 explains why deterministic single-goal regression is fundamentally mismatched to partially observable long-horizon exploration: under multimodal futures, the Bayes-optimal deterministic predictor is mean-seeking and may become brittle or infeasible. Proposition 3.2 further shows that, once the learning target is reformulated as a conditional anchor distribution, CFM provides a principled way to learn its transport field in the low-dimensional anchor space. This justifies our design choice of combining multimodal anchor modeling with downstream explicit planning.

B.2 Training and Inference Details

This subsection provides the full training objectives, staged training procedure, and commitment/replanning inference strategy used in our implementation.

For completeness, we first summarize the overall loss decomposition:

$$\mathcal{L} = \mathcal{L}_{\text{obs}} + \mathcal{L}_{\text{anchor}} + \mathcal{L}_{\text{sel}}, \quad (33)$$

where $\mathcal{L}_{\text{obs}}$, $\mathcal{L}_{\text{anchor}}$, and $\mathcal{L}_{\text{sel}}$ correspond to obstacle-map supervision, multimodal anchor generation, and hierarchical selection, respectively. The obstacle loss $\mathcal{L}_{\text{obs}}$ supervises the obstacle map prediction, and its implementation details are provided in Appendix B.3.3.

The anchor-generation loss is

$$\mathcal{L}_{\text{anchor}} = \mathcal{L}_{\text{FM}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}, \quad (34)$$

where $\mathcal{L}_{\text{FM}}$ is the conditional flow-matching objective and $\mathcal{L}_{\text{div}}$ is the diversity regularization term used to discourage anchor collapse. Its implementation is detailed in Appendix B.3.4.

The hierarchical-selection loss combines a mode-classification term, an intra-mode ranking term, and several auxiliary supervision terms for gain, risk, cost, continuity, and uncertainty estimation:

$$\mathcal{L}_{\text{sel}} = \mathcal{L}_{\text{mode}} + \mathcal{L}_{\text{rank}} + \lambda_{\text{aux}}^{\text{sel}} \mathcal{L}_{\text{aux}}^{\text{sel}}. \quad (35)$$

For readability, we defer the fully expanded form and the exact definitions of these auxiliary terms to Appendix B.5.4.

B.2.1 Staged Training

Because context encoding, anchor generation, and hierarchical selection exhibit significant differences in their optimization objectives, we adopt staged training to avoid unstable optimization caused by excessive coupling between modules in the early stages of training. The overall process is divided into four stages. **(I) Context warm-up:** We train the shared encoder f_ϕ and obstacle prediction head g_ψ , while optionally retaining the auxiliary value-map head h_ν as an additional supervision branch during training, to obtain a robust context representation for open environments. **(II) Anchor flow training:** With the context encoder fixed or partially frozen, we utilize the oracle anchor set to train the CFM module v_θ , learning the conditional anchor distribution $p_\theta(a | z_t)$. **(III) Hierarchical selector training:** Based on the candidate anchors generated by CFM, planned paths, and mode clustering results, we train the mode scorer s_η and local reranker. **(IV) Joint fine-tuning:** Finally, we jointly fine-tune all the aforementioned modules to enhance overall synergistic performance.

To support Stage II and Stage III, we construct an oracle path bank, oracle anchor set, and oracle mode labels for each training state. Given a training state $(E_t, c_t, \mathcal{P}_t)$, we generate a set of high-gain and diverse candidate paths on the ground-truth obstacle map O_t^{GT} , and calculate their actual accumulated coverage gain $\Delta \text{Cov}(\tau)$. We then extract the corresponding anchors to form the oracle anchor set $\mathcal{A}_t^+$, and apply the same clustering as in Sec. 3.3 to the oracle paths to form the oracle mode set $\{\mathcal{M}_t^{+,m}\}$. The mode containing the optimal path is denoted as the positive mode m_t^+ , and the optimal path itself is denoted as τ_t^+ . The overall training pseudocode is shown in Algorithm 1.

Algorithm 1 Staged Training

Require:

Training set $\mathcal{D}_{\text{tr}}$, maximum epochs for each stage $\{E_{\text{ctx}}, E_{\text{anchor}}, E_{\text{sel}}, E_{\text{joint}}\}$, optimizer(s), and loss weights.

Ensure:

Trained parameters $\{\phi, \psi, \theta, \eta\}$ and optional auxiliary-head parameters ν .

1: **// Stage I: Context warm-up**

2: **for** epoch = 1, ..., E_{ctx} **do**

3: Sample mini-batch $\mathcal{B} = \{(E_t, O_t^{\text{GT}}, M_t^{\text{GT}})\}$ from $\mathcal{D}_{\text{tr}}$.

```

509 4:   Encode context  $z_t = f_\phi(E_t)$  using Sec. 3.1.
510 5:   Predict obstacle map  $\hat{O}_t = g_\psi(z_t)$ .
511 6:   Optionally predict auxiliary value map  $\hat{M}_t^{\text{aux}} = h_\nu(z_t)$  for auxiliary supervision only.
512 7:   Compute context loss  $\mathcal{L}_{\text{ctx}}$  following Appendix B.2.
513 8:   Update  $\{\phi, \psi\}$ , and optionally  $\nu$ , by minimizing  $\mathcal{L}_{\text{ctx}}$ .
514 9: end for
515 10: // Stage II: Anchor flow training
516 11: Freeze or partially freeze  $\{\phi, \psi\}$ .
517 12: for epoch = 1, ...,  $E_{\text{anchor}}$  do
518 13:   Sample mini-batch  $\mathcal{B} = \{(E_t, c_t, \mathcal{A}_t^+)\}$  from  $\mathcal{D}_{\text{tr}}$ .
519 14:   Encode context  $z_t = f_\phi(E_t)$ .
520 15:   Sample  $x_0 \sim \mathcal{N}(0, I)$ ,  $x_1 \sim p_1(\cdot | z_t)$ , and  $t \sim \mathcal{U}(0, 1)$  according to Eq. 6.
521 16:   Interpolate  $x_t = (1 - t)x_0 + tx_1$  using Eq. 7, and set target velocity  $u_t = x_1 - x_0$ .
522 17:   Predict conditional velocity  $v_\theta(x_t, t, z_t)$  and optimize the flow-matching objective in Eq. 9.
523 18:   Compute anchor loss  $\mathcal{L}_{\text{anchor}} = \mathcal{L}_{\text{FM}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}$ , where the diversity term is defined in Eq. 12.
524
525 19:   Update  $\theta$  by minimizing  $\mathcal{L}_{\text{anchor}}$ .
526 20: end for
527 21: // Stage III: Hierarchical selector training
528 22: Freeze or partially freeze  $\{\phi, \psi, \theta\}$ .
529 23: for epoch = 1, ...,  $E_{\text{sel}}$  do
530 24:   Sample mini-batch  $\mathcal{B} = \{(E_t, c_t, m_t^+, \tau_t^+, \text{oracle labels})\}$  from  $\mathcal{D}_{\text{tr}}$ .
531 25:   Encode context  $z_t = f_\phi(E_t)$ .
532 26:   Predict obstacle map  $\hat{O}_t = g_\psi(z_t)$ .
533 27:   Sample anchor set  $\mathcal{A}_t = \{a_t^{(k)}\}_{k=1}^K$  from  $p_\theta(a | z_t)$  using Eq. 11.
534 28:   for each anchor  $a_t^{(k)} \in \mathcal{A}_t$  do
535 29:     Plan candidate path  $\tau_t^{(k)} = \text{Plan}(c_t, a_t^{(k)}, \hat{O}_t)$  using Eq. 13.
536 30:     Compute path descriptor  $\varphi(\tau_t^{(k)})$  using Eq. 14.
537 31:   end for
538 32:   Cluster candidate paths into modes  $\{\mathcal{M}_t^{(m)}\}_{m=1}^{M_t}$  according to Eq. 15 and Eq. 16.
539 33:   Build each mode representation  $z_t^{(m)} = \Gamma(z_t, \hat{\tau}_t^{(m)}, \chi_t^{(m)})$  using Eq. 18, where  $\chi_t^{(m)}$  is defined
540   in Eq. 17.
541 34:   Compute mode heads  $(G_m, R_m, U_m, C_m, D_m) = s_\eta(z_t^{(m)})$  via Eq. 19, and compute mode
542   scores  $S_m^{\text{mode}}$  via Eq. 20.
543 35:   Compute local representations  $\zeta_t(\tau) = \Lambda(z_t, \tau, \varphi(\tau), \chi_t^{(m_t^+)})$  using Eq. 22.
544 36:   Compute local path scores  $S^{\text{local}}(\tau)$  within the positive mode using Eq. 23.
545 37:   Compute selection loss  $\mathcal{L}_{\text{sel}}$  following Sec. 3.4.
546 38:   Update selector-related parameters  $\eta$  (and optional reranker parameters) by minimizing  $\mathcal{L}_{\text{sel}}$ .
547 39: end for
548 40: // Stage IV: Joint fine-tuning
549 41: Unfreeze all trainable modules.
550 42: for epoch = 1, ...,  $E_{\text{joint}}$  do
551 43:   Sample mini-batch  $\mathcal{B}$  from  $\mathcal{D}_{\text{tr}}$ .
552 44:   Compute the total loss by jointly combining  $\mathcal{L}_{\text{ctx}}$ ,  $\mathcal{L}_{\text{anchor}}$ , and  $\mathcal{L}_{\text{sel}}$ .
553 45:   Jointly update  $\{\phi, \psi, \theta, \eta\}$ , while optionally updating  $\nu$  only through the auxiliary supervision
554   branch.
555 46: end for
556 47: return Trained parameters  $\{\phi, \psi, \theta, \eta\}$  and optional auxiliary-head parameters  $\nu$ .

```

557 B.2.2 Commitment and Replanning Inference

558 During the inference phase, we adopt a partial commitment mechanism. Let the path prefix
559 already submitted for execution be $\bar{\tau}_{t-1}$, and its remaining unexecuted steps be denoted as b_t . If the
560 current committed path is still valid, the system directly executes the next step along it. Specifically,
561 replanning is triggered if and only if any of the following four conditions are met. **(I) Plan exhausted:**
562 The submitted path prefix has been fully executed. **(II) Path blocked:** New obstacles render the
563 current path invalid. **(III) Severe deviation:** The actual executed pose significantly deviates from

the submitted path. **(IV) Gain collapse:** The expected gain of the current submitted path drops significantly under the latest context.

When replanning is required, the system executes the complete hierarchical inference pipeline: first encoding the context, then generating the anchor set $\mathcal{A}_t$, planning candidate paths for each anchor and clustering them into a mode set $\{\mathcal{M}_t^{(m)}\}$, followed by mode-level selection and intra-mode local reranking to obtain a long-horizon path τ_t^* . Finally, only the first L_c steps of this path are submitted as the new committed path prefix. The overall inference pseudocode is shown in Algorithm 2.

Algorithm 2 Commitment/Replanning Inference

Require:

Current state $(E_t, c_t, \text{history}, \bar{\tau}_{t-1}, b_t)$, trained model $\{f_\phi, g_\psi, p_\theta, s_\eta, \text{local reranker}\}$, commitment length L_c .

Ensure:

Next action c_{t+1} or a new committed path prefix.

1: **// Step 1: Check whether replanning is needed**

2: **if** $b_t > 0$ and current committed path $\bar{\tau}_{t-1}$ is valid **then**

3: **if not** (plan exhausted $\vee$ path blocked $\vee$ severe deviation $\vee$ gain collapse) **then**

4: Execute the next step on $\bar{\tau}_{t-1}$ and decrement b_t .

5: **return** c_{t+1}

6: **end if**

7: **end if**

8: **// Step 2: Recompute current context**

9: Compute $z_t = f_\phi(E_t)$ and $\hat{O}_t = g_\psi(z_t)$.

10: **// Step 3: Sample multimodal anchor set**

11: Sample $\mathcal{A}_t = \{a_t^{(k)}\}_{k=1}^K$ from $p_\theta(a | z_t)$ using Eq. 11.

12: **// Step 4: Planning and path construction**

13: Initialize candidate path set $\mathcal{T}_t = \emptyset$.

14: **for** each anchor $a_t^{(k)} \in \mathcal{A}_t$ **do**

15: Compute $\tau_t^{(k)} = \text{Plan}(c_t, a_t^{(k)}, \hat{O}_t)$ using Eq. 13.

16: **if** $\tau_t^{(k)}$ is feasible **then**

17: Compute path descriptor $\varphi(\tau_t^{(k)})$ using Eq. 14.

18: Add $\tau_t^{(k)}$ to $\mathcal{T}_t$.

19: **end if**

20: **end for**

21: **// Step 5: Mode construction**

22: Cluster $\mathcal{T}_t$ into $\{\mathcal{M}_t^{(m)}\}_{m=1}^{M_t}$ using Eq. 15 and Eq. 16.

23: Compute representative path $\tilde{\tau}_t^{(m)}$ and mode statistics $\chi_t^{(m)}$ using Eq. 17.

24: **// Step 6: Mode-level scoring**

25: **for** each mode $\mathcal{M}_t^{(m)}$ **do**

26: Build mode representation $z_t^{(m)} = \Gamma(z_t, \tilde{\tau}_t^{(m)}, \chi_t^{(m)})$ using Eq. 18.

27: Compute mode heads $(G_m, R_m, U_m, C_m, D_m) = s_\eta(z_t^{(m)})$ using Eq. 19.

28: Compute mode score $S_m^{\text{mode}} = w_G G_m - w_R R_m - w_U U_m - w_C C_m + w_D D_m$ using Eq. 20.

29: **end for**

30: Select best mode $m_t^* = \arg \max_m S_m^{\text{mode}}$ using Eq. 21.

31: **// Step 7: Intra-mode local reranking**

32: **for** each path $\tau \in \mathcal{M}_t^{(m_t^*)}$ **do**

33: Build local representation $\zeta_t(\tau) = \Lambda(z_t, \tau, \varphi(\tau), \chi_t^{(m_t^*)})$ using Eq. 22.

34: Predict path-level heads $(\hat{G}(\tau), \hat{R}(\tau), \hat{C}(\tau), \hat{H}(\tau))$.

35: Compute local score $S^{\text{local}}(\tau) = \hat{G}(\tau) - \lambda_R \hat{R}(\tau) - \lambda_C \hat{C}(\tau) - \lambda_H \hat{H}(\tau)$ using Eq. 23.

36: **end for**

37: Select final path $\tau_t^* = \arg \max_{\tau \in \mathcal{M}_t^{(m_t^*)}} S^{\text{local}}(\tau)$ using Eq. 24.

38: **// Step 8: Partial commitment**

39: Commit only the first L_c steps: $\bar{\tau}_t = \text{Prefix}(\tau_t^*, L_c)$.

40: Set remaining commitment budget $b_t = \text{length}(\bar{\tau}_t)$.

41: **// Step 9: Execute**

618 42: **return** The first step of $\bar{\tau}_t$ as c_{t+1} .

619 In practical implementations, we found that staged training is particularly important for optimiza-
620 tion stability: if all modules are jointly optimized end-to-end directly in the early stages of training,
621 an unstable coupling tends to form between the distribution generated by CFM and the subsequent
622 mode selector, leading to a simultaneous degradation in both mode clustering quality and selector
623 supervision signals. In contrast, obtaining a more stable context representation via $\mathcal{L}_{\text{ctx}}$ first, and
624 then learning the anchor distribution and hierarchical selection, can significantly enhance training
625 convergence and cross-difficulty generalization performance.

626 B.3 Network and Implementation Details

627 This subsection presents the detailed network architecture, implementation settings, and default
628 hyperparameter configurations used in all experiments.

629 B.3.1 Dataset and simulation setup.

630 We evaluate our method on Matterport3D (MP3D) [17] and AiMDoom [8]. For MP3D, we follow
631 the same protocol as prior work [8] for fair comparison. The input posed depth images have a
632 resolution of 256×256 with a horizontal field of view (hFOV) of 90° . The mobile agent starts from
633 the traversable space at a height of 1.25m and selects the next camera pose by either moving forward
634 by 6.5cm or turning left/right by 10° . Depending on scene size, the maximum episode length is set to
635 1000 or 2000 steps. Following [8], we consider single-floor scenes only, with 10 scenes for training
636 and 5 scenes for evaluation.

637 For AiMDoom, we adopt a 70/30 train/test split for scenes within each difficulty level. The input
638 RGB-D images are rendered at 456×256 resolution with hFOV 90° . The agent moves in a traversable
639 space at height 1.65m, with 4 translational actions (forward, backward, left, and right by 1.5m) and 8
640 rotational actions (left/right turns in increments of 45° , covering the full 360°). To support dense
641 reconstruction, all methods capture three additional interpolated views between adjacent poses. The
642 maximum number of decision steps is set to 100, 200, 400, and 500 for the Simple, Normal, Hard,
643 and Insane levels, respectively.

644 **Evaluation metrics.** On AiMDoom, we adopt two standard metrics for active 3D mapping: (1)
645 *Final Coverage*, which measures the final scene coverage at the end of an episode, and (2) *AUC*, which
646 evaluates reconstruction efficiency by the area under the coverage curve over time. Surface coverage
647 is computed against the ground-truth mesh following prior work [8]. We evaluate five trajectories per
648 scene using identical random initial poses for different methods, and report the mean and standard
649 deviation over all testing trajectories. For MP3D, in addition to the above active-mapping metrics, we
650 also report the completion metrics used in prior work: (1) *Comp. (%)*, the percentage of ground-truth
651 vertices within 5cm of any observation, and (2) *Comp. (cm)*, the average minimum distance from
652 each ground-truth vertex to the observed reconstruction.

653 B.3.2 Implementation details.

654 Unless otherwise specified, we adopt the same mapping-state construction and training protocol
655 as [8] for fair comparison. The model takes as input a stack of $K = 4$ projected 2D occupancy images
656 together with one projected trajectory-history image, each of resolution 256×256 and covering a
657 $40\text{m} \times 40\text{m}$ exploration area centered at the current camera pose. The encoder produces a feature
658 tensor of size $16 \times 16 \times 1024$. The obstacle map has size $256 \times 256 \times 1$, both corresponding to the
659 same $40\text{m} \times 40\text{m}$ local region.

660 We train the model for at most $N = 15$ iterations. During the first $N_e = 1$ iteration, we use easier
661 training samples and collect $S_n = 2$ trajectories per scene; in later iterations, we use all available
662 samples and reduce the number of trajectories to $S_n = 1$ per scene. Each trajectory has length 100
663 and starts from a random location. During the first data-collection iteration, we randomly sample 1000
664 validation examples from memory and exclude them from training. We use gradient accumulation,
665 resulting in an effective batch size of 448. Early stopping is applied when the validation loss no
666 longer decreases. All experiments are conducted on two 48GB GPUs.

667 B.3.3 State Representation, Context Encoder, and Dense Prediction Heads

668 **State Representation Construction.** Given the accumulated reconstructed point cloud $\mathcal{P}_t$, the
669 implementation constructs the network input through `build_state_input(...)`. In code, the
670 accumulated point cloud is stored as `full_pc`, which is updated online by concatenating local
671 point clouds back-projected from depth maps at each step. Specifically, after each depth infer-
672 ence, `compute_partial_point_cloud(...)` is called to obtain the local point cloud part, and

673 full_pc is updated by vertically stacking part onto the existing global point set. The resulting
 674 full_pc is then divided into K horizontal slices along the height axis using predefined bound-
 675 aries `y_bins` and `torch.bucketize`. For each slice, the corresponding points are transformed
 676 into the ego-centric coordinate system of the current camera pose and projected onto a 2D map,
 677 yielding K point-cloud projection channels $I_{t,1}^{pc}, \dots, I_{t,K}^{pc}$. In parallel, the historical camera trajectory
 678 `x_cam_history` is transformed in the same way and rasterized into a visitation-frequency map I_t^{hist} .
 679 The final state representation is therefore

$$E_t = \{I_{t,1}^{pc}, \dots, I_{t,K}^{pc}, I_t^{\text{hist}}\}, \quad (36)$$

680 which is concatenated along the channel dimension into a single tensor before being fed into the
 681 network. Under the default configuration, `flow_n_pieces` = 4, so the input has $K + 1 = 5$
 682 channels.

683 **Context Encoder.** The shared context encoder f_ϕ is implemented as `MappingProgressEncoder`,
 684 which takes the multi-channel state representation E_t as input and produces both a global latent
 685 representation and intermediate multi-scale feature maps. Concretely, the encoder consists of a stem
 686 block followed by three downsampling convolutional stages, forming a hierarchical feature pipeline
 687 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$. As the depth increases, the receptive field gradually expands and the features
 688 become more semantically abstract. The deepest feature map x_4 is then processed by global average
 689 pooling, a linear projection, `LayerNorm`, and `ReLU` to obtain the final latent vector

$$z_t = f_\phi(E_t), \quad z_t \in \mathbb{R}^{256}. \quad (37)$$

690 In implementation, the encoder returns not only the latent z_t but also the intermediate feature maps
 691 $\{x_1, x_2, x_3, x_4\}$, which are later used by the dense obstacle decoder through skip connections.

692 **Obstacle Prediction Head.** The obstacle prediction head g_ψ is implemented as
 693 `ObstacleMapDecoder`. Rather than decoding only from the global latent, this branch follows
 694 a U-Net-style design built on the multi-scale encoder features. Starting from the deepest feature, it
 695 progressively upsamples the representation and fuses it with encoder features at matching spatial
 696 resolutions via skip connections, thereby recovering fine-grained spatial structure while preserving
 697 high-level context. The final output is produced by a single-channel convolution followed by a
 698 sigmoid activation, yielding an obstacle probability map

$$\hat{O}_t = g_\psi(z_t), \quad \hat{O}_t \in [0, 1]^{H_o \times W_o}. \quad (38)$$

699 Thus, although we write the head as $g_\psi(z_t)$ at the method level, its actual implementation is a dense
 700 multi-scale decoder conditioned on the encoder feature hierarchy.

701 **Auxiliary Value Head.** Following prior work, we optionally retain an auxiliary value-map
 702 head h_ν as an additional supervision branch during training. This module is implemented as
 703 `ValueAuxDecoder`. Unlike the obstacle branch, it does not use a U-Net-style feature hierarchy;
 704 instead, it starts directly from the global latent z_t , passes through a two-layer multilayer perceptron,
 705 and outputs a dense multi-channel map:

$$\hat{M}_t^{\text{aux}} = h_\nu(z_t), \quad \hat{M}_t^{\text{aux}} \in \mathbb{R}^{C_v \times H_v \times W_v}. \quad (39)$$

706 Under the default setting, its output shape is $8 \times 64 \times 64$. This branch is used only as an auxiliary
 707 training signal and is not involved in inference or downstream planning.

708 B.3.4 Conditional Flow Matching Implementation

709 This section complements the specific implementation of Conditional Flow Matching, correspond-
 710 ing to `CFMAnchorProposer` in the project.

711 **Conditional Velocity Field.** The conditional velocity field:

$$v_\theta(x_t, t, z_t) \quad (40)$$

712 is parameterized by `CFMAnchorProposer`. Its inputs include the noisy anchor x_t , the time variable t ,
 713 and the context latent z_t , which are mapped into a shared hidden space through three independent
 714 branches:

$$\mathbf{h}_a = \text{Proj}_a(x_t), \quad \mathbf{h}_t = \text{Proj}_t(\text{TimeEmbed}(t)), \quad \mathbf{h}_z = \text{Proj}_z(z_t). \quad (41)$$

715 Subsequently, these three are concatenated and passed through an MLP to output the velocity
 716 prediction in the anchor space:

$$v_\theta(x_t, t, z_t) = \text{MLP}([\mathbf{h}_a, \mathbf{h}_t, \mathbf{h}_z]). \quad (42)$$

717 In the current implementation, `anchor_dim = 4` and `hidden_dim = 256`. Therefore, CFM is not
 718 an abstract symbolic function in the code, but a conditional velocity field parameterized by an MLP.
 719 **Sinusoidal Time Embedding.** The time variable t is encoded via `SinusoidalTimeEmbedding`.
 720 In implementation, if the embedding dimension is set to d , a decreasing frequency sequence $\{\omega_\ell\}$ is
 721 first constructed, and then the following is outputted:

$$\text{TimeEmbed}(t) = [\sin(\omega_1 t), \dots, \sin(\omega_{d/2} t), \cos(\omega_1 t), \dots, \cos(\omega_{d/2} t)]. \quad (43)$$

722 This time embedding is subsequently projected to the hidden dimension via a linear layer and ReLU,
 723 to be fused with the anchor features and latent features.

724 **Flow Matching Loss.** The training loss is implemented by `flow_matching_loss(...)`. Given
 725 the `target_anchor`, Gaussian noise is first sampled:

$$x_0 \sim \mathcal{N}(0, I), \quad (44)$$

726 then time is sampled from a uniform distribution:

$$t \sim \mathcal{U}(0, 1), \quad (45)$$

727 and the linear interpolation state is constructed:

$$x_t = (1 - t)x_0 + tx_1, \quad x_1 = \text{target_anchor}. \quad (46)$$

728 The target velocity is defined as:

$$u_t = x_1 - x_0. \quad (47)$$

729 After the model predicts the velocity, the Mean Squared Error is used for supervision:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t, z_t) - u_t\|_2^2]. \quad (48)$$

730 **Oracle Anchor Construction.** In the project implementation, the training target for CFM does not
 731 directly come from an explicit probability density, but from an oracle anchor. Specifically, the main
 732 model first obtains the latent via the context encoder, and then calls `flow_matching_loss(...)` via
 733 `anchor_loss(...)`, where the target anchor comes from the endpoint parameterized representation
 734 corresponding to the high-gain trajectory. In other words, the conditional distribution:

$$p_\theta(a_t | E_t) \quad (49)$$

735 is implicitly modeled in the implementation through:

$$z_t = f_\phi(E_t) \quad \text{and} \quad \text{CFMAnchorProposer}. \quad (50)$$

736 **Anchor Sampling.** During the inference phase, anchor sampling is implemented by `sample(...)`.
 737 Given the latent z_t , the latent is first repeated to support parallel sampling of multiple candidate
 738 anchors, which are then initialized from Gaussian noise:

$$x^{(0)} \sim \mathcal{N}(0, I). \quad (51)$$

739 Subsequently, explicit Euler integration is adopted:

$$x^{(i+1)} = x^{(i)} + v_\theta(x^{(i)}, t_i, z_t) \Delta t, \quad \Delta t = \frac{1}{\text{steps}}. \quad (52)$$

740 where the number of integration steps is controlled by `steps`, and the initial noise scale is controlled
 741 by `temperature`. After integration concludes, parameter space constraints are further applied to the
 742 sampling results: The first 2 positional components are clamped to $[-1, 1]$; The middle 2 directional
 743 components are normalized to unit vectors; The final dimension is mapped to $(0, 1)$ via sigmoid.
 744 Ultimately, the multimodal anchor set is obtained:

$$\mathcal{A}_t = \{a_t^{(1)}, \dots, a_t^{(K)}\}. \quad (53)$$

745 B.4 Detailed Definitions of Path Features and Distance Metrics

746 B.4.1 Path-Level Descriptors

747 The following path-level descriptors are computed from candidate paths, obstacle maps, auxiliary
 748 value maps, and historical execution trajectories via hand-crafted structural rules. We first formulate
 749 the representation of the k -th candidate path:

$$\tau_t^{(k)} = \{(p_{k,\ell}, \theta_{k,\ell})\}_{\ell=1}^{L_k}, \quad (54)$$

750 where: $p_{k,\ell} \in \mathbb{R}^2$: the position coordinates of the ℓ -th discrete point on the path; $\theta_{k,\ell}$: the orientation
751 of the path point; L_k : the discrete number of steps of the path (i.e., path length).

752 For the planned position sequence $\{p_{k,\ell}\}$, we define the directional angle of the difference vector
753 between adjacent positions as the orientation of intermediate path points. Let

$$p_{k,\ell} = \begin{pmatrix} p_{k,\ell}^x, p_{k,\ell}^y \end{pmatrix}, \quad (55)$$

754 then the orientation is derived as:

$$\theta_{k,\ell} = \text{atan2} \left(p_{k,\ell+1}^y - p_{k,\ell}^y, p_{k,\ell+1}^x - p_{k,\ell}^x \right), \quad (56)$$

755 and the endpoint orientation θ_{k,L_k} is provided by the anchor orientation $a_{t,\theta}^{(k)}$.

756 Simultaneously, we define the following auxiliary map structures and variables: $\hat{O}_t(x) \in [0, 1]$:
757 the obstacle probability at position x ; $\hat{M}_t^{\text{aux}}(x, \theta)$: the predicted value of the auxiliary value map at
758 pose (x, θ) ; $H_t(x) \in [0, 1]$: the historical visitation map, characterizing the frequency (normalized
759 heat) that position x has been visited by past trajectories; τ_{t-1} : the path submitted for execution in
760 the previous round (utilized for continuity calculation); $\mathcal{E}_t(x)$: the obstacle map entropy, quantifying
761 the uncertainty of the obstacle prediction, defined as:

$$\mathcal{E}_t(x) = -\hat{O}_t(x) \log \hat{O}_t(x) - (1 - \hat{O}_t(x)) \log(1 - \hat{O}_t(x)). \quad (57)$$

762 Based on the above definitions, the specific calculation methods for the six path-level descriptors
763 are formulated as follows:

764 **Expected Gain Proxy g_k .** By reusing the semantic output of the auxiliary value map, we directly
765 characterize the long-horizon coverage gain potential of the path. The auxiliary value at the endpoint
766 pose of the path serves as the gain proxy:

$$g_k = \hat{M}_t^{\text{aux}}(p_{k,L_k}, \theta_{k,L_k}). \quad (58)$$

767 **Path Risk r_k .** This metric comprehensively considers the probability of the path traversing
768 obstacles and the obstacle clearance, avoiding collision risks in narrow passage scenarios:

$$r_k = \alpha_r \cdot \frac{1}{L_k} \sum_{\ell=1}^{L_k} \hat{O}_t(p_{k,\ell}) + (1 - \alpha_r) \cdot \frac{1}{d_{\min}(\tau_t^{(k)}) + \varepsilon}, \quad (59)$$

769 where

$$d_{\min}(\tau_t^{(k)}) = \min_{\ell} \text{DistToObstacle}(p_{k,\ell}) \quad (60)$$

770 represents the minimum safe distance from the path to obstacles, $\varepsilon > 0$ is a small constant to prevent
771 division by zero, and $\alpha_r \in [0, 1]$ is a weight balancing the obstacle probability and the safety distance.
772 The first term measures the “traversal risk,” and the second term measures the “proximity risk.” Their
773 combination better aligns with the navigation requirements in open environments.

774 **Path Cost c_k .** This descriptor integrates path length and steering smoothness to penalize inefficient
775 paths:

$$c_k = \alpha_c \cdot \frac{L_k}{L_{\max}} + (1 - \alpha_c) \cdot \frac{1}{L_k - 1} \sum_{\ell=1}^{L_k-1} \frac{|\text{wrap}(\theta_{k,\ell+1} - \theta_{k,\ell})|}{\pi}, \quad (61)$$

776 where: $L_{\max}$: the predefined maximum path length (used for normalization); $\text{wrap}(\cdot)$: an angle
777 difference mapping function that constrains the difference within $(-\pi, \pi]$; $\alpha_c \in [0, 1]$: a weight
778 balancing the length cost and the steering cost. From a physical perspective, a larger c_k implies a
779 longer path or more frequent steering, resulting in lower execution efficiency.

780 **Path Novelty n_k .** This term penalizes repeated visits to historical regions and encourages the
781 exploration of uncovered spaces:

$$n_k = 1 - \frac{1}{L_k} \sum_{\ell=1}^{L_k} H_t(p_{k,\ell}). \quad (62)$$

782 If the path primarily passes through regions with high visitation heat, n_k approaches 0; if it primarily
783 explores low-visitation areas, n_k approaches 1.

784 **Continuity Deviation** h_k . This descriptor penalizes severe deviations from the previously executed
 785 path, avoiding decision oscillations. Let the path submitted in the previous round be:

$$\bar{\tau}_{t-1} = \{(\bar{p}_\ell, \bar{\theta}_\ell)\}_{\ell=1}^{L_c}, \quad (63)$$

786 then the continuity deviation is calculated as:

$$h_k = \alpha_h \cdot \frac{1}{L_c} \sum_{\ell=1}^{L_c} \|p_{k,\ell} - \bar{p}_\ell\|_2 + (1 - \alpha_h) \cdot \frac{1}{L_c} \sum_{\ell=1}^{L_c} (1 - \cos(\theta_{k,\ell} - \bar{\theta}_\ell)), \quad (64)$$

787 where: The first term is the average Euclidean distance of the position deviation; The second term
 788 is the cosine similarity of the orientation deviation; $\alpha_h \in [0, 1]$ is a weight balancing position and
 789 orientation deviations.

790 **Uncertainty Statistic** u_k . This characterizes the obstacle prediction uncertainty in the regions
 791 traversed by the path, preventing aggressive decisions in highly ambiguous areas:

$$u_k = \frac{1}{L_k} \sum_{\ell=1}^{L_k} \mathcal{E}_t(p_{k,\ell}). \quad (65)$$

792 A larger u_k indicates higher uncertainty in the obstacle prediction for the regions the path traverses,
 793 leading to poorer execution stability, which necessitates cautious selection, especially in open-
 794 environment scenarios.

795 B.4.2 Path Distance

796 This section details the four components of the path distance defined in the main text:

$$d(\tau_i, \tau_j) = \lambda_1 d_{\text{end}}(\tau_i, \tau_j) + \lambda_2 d_{\text{shape}}(\tau_i, \tau_j) + \lambda_3 d_{\text{prefix}}(\tau_i, \tau_j) + \lambda_4 d_{\text{topo}}(\tau_i, \tau_j), \quad (66)$$

797 which respectively characterize the consistency of the endpoint position, overall geometric shape,
 798 prefix execution behavior, and obstacle circumvention topology.

799 To achieve effective clustering in the path space, it is necessary to first unify the metric standard
 800 for any two paths, and then define the distance from the four dimensions. The core premise is to
 801 resample all paths to a fixed length S to eliminate metric biases caused by differences in original path
 802 lengths. The resampled paths are represented as:

$$\bar{\tau}_i = \{(\bar{p}_{i,s}, \bar{\theta}_{i,s})\}_{s=1}^S, \quad \bar{\tau}_j = \{(\bar{p}_{j,s}, \bar{\theta}_{j,s})\}_{s=1}^S, \quad (67)$$

803 where $\bar{p}_{i,s} \in \mathbb{R}^2$ and $\bar{\theta}_{i,s}$ denote the position and orientation of the s -th point of the resampled path
 804 $\bar{\tau}_i$, respectively.

805 **Endpoint Distance** d_{end} . Measures the similarity of the final goal poses (position and orientation)
 806 of the two paths:

$$d_{\text{end}}(\tau_i, \tau_j) = \|\bar{p}_{i,S} - \bar{p}_{j,S}\|_2 + \lambda_{\theta}^{\text{end}} \cdot (1 - \cos(\bar{\theta}_{i,S} - \bar{\theta}_{j,S})), \quad (68)$$

807 where $\lambda_{\theta}^{\text{end}} \geq 0$ is a weight balancing endpoint position and orientation differences.

808 **Shape Distance** d_{shape} . Measures the similarity of the overall geometric shapes of the two paths
 809 after resampling, utilizing the point-wise average Euclidean distance:

$$d_{\text{shape}}(\tau_i, \tau_j) = \frac{1}{S} \sum_{s=1}^S \|\bar{p}_{i,s} - \bar{p}_{j,s}\|_2. \quad (69)$$

810 **Prefix Distance** d_{prefix} . Specifically characterizes the consistency of the preceding execution
 811 behavior of the paths, preventing paths with “similar endpoints but completely different starting
 812 actions” from being misjudged as similar. Given a prefix length $S_p < S$ (e.g., $S_p = S/3$), then:

$$d_{\text{prefix}}(\tau_i, \tau_j) = \frac{1}{S_p} \sum_{s=1}^{S_p} \|\bar{p}_{i,s} - \bar{p}_{j,s}\|_2. \quad (70)$$

813 **Topological Distance** d_{topo} . The core idea is to measure the consistency of circumvention methods
 814 through the overlap of the dilated path masks: Path rasterization: map the resampled path $\bar{\tau}$ to a

815 rasterized binary mask; Mask dilation: perform moderate morphological dilation on the binary mask
 816 to obtain the path “corridor mask” $B(\tau)$; Topological distance definition:

$$d_{\text{topo}}(\tau_i, \tau_j) = 1 - \text{IoU}(B(\tau_i), B(\tau_j)), \quad (71)$$

817 where $\text{IoU}(\cdot, \cdot)$ is the Intersection over Union. If two paths navigate around obstacles along the same
 818 corridor and on the same side, then $B(\tau_i)$ and $B(\tau_j)$ highly overlap, making d_{topo} approach 0; if the
 819 paths circumvent via different passages, different room entrances, or on opposite sides of obstacles,
 820 d_{topo} approaches 1.

821 **Overall Path Distance.** Integrating the above four dimensions, the overall path distance is:

$$d(\tau_i, \tau_j) = \lambda_1 d_{\text{end}}(\tau_i, \tau_j) + \lambda_2 d_{\text{shape}}(\tau_i, \tau_j) + \lambda_3 d_{\text{prefix}}(\tau_i, \tau_j) + \lambda_4 d_{\text{topo}}(\tau_i, \tau_j), \quad (72)$$

822 where $\lambda_1, \lambda_2, \lambda_3, \lambda_4 \geq 0$ are predefined balancing weights.

823 B.4.3 Mode Statistics

824 This section details the mode statistics defined in the main text:

$$\chi_t^{(m)} = \left[|\mathcal{M}_t^{(m)}|, \bar{g}_m, \bar{r}_m, \text{Var}(g)_m, \text{Var}(r)_m, \rho_m \right], \quad (73)$$

825 which are utilized to characterize the overall scale, gain level, risk level, and internal dispersion of
 826 each exploration mode. Given the m -th mode:

$$\mathcal{M}_t^{(m)} = \{\tau_k\}, \quad (74)$$

827 the components of its statistics are defined as follows:

828 **Mode Size.** Defined as the number of candidate paths within the mode:

$$|\mathcal{M}_t^{(m)}|. \quad (75)$$

829 It reflects how many paths in the current candidate set are classified into the same class of long-
 830 horizon exploration behavior. Generally, a larger mode indicates that this exploration strategy is more
 831 common among the current anchor sampling results.

832 **Mean Gain and Mean Risk.** The mean gain and mean risk of a mode are respectively defined as:

$$\bar{g}_m = \frac{1}{|\mathcal{M}_t^{(m)}|} \sum_{\tau_k \in \mathcal{M}_t^{(m)}} g_k, \quad \bar{r}_m = \frac{1}{|\mathcal{M}_t^{(m)}|} \sum_{\tau_k \in \mathcal{M}_t^{(m)}} r_k. \quad (76)$$

833 where g_k and r_k are the path-level gain proxy and path-level risk descriptor defined in Appendix
 834 B.4.1. $\bar{g}_m$ reflects the overall long-horizon exploration potential of the mode, while $\bar{r}_m$ reflects the
 835 overall safety and executability of the mode.

836 **Gain Variance and Risk Variance.** The dispersion of gain and risk within a mode is defined as:

$$\text{Var}(g)_m = \text{Var}_{\tau_k \in \mathcal{M}_t^{(m)}}(g_k), \quad \text{Var}(r)_m = \text{Var}_{\tau_k \in \mathcal{M}_t^{(m)}}(r_k). \quad (77)$$

837 These two quantities measure the consistency of path quality within the same mode. If $\text{Var}(g)_m$ or
 838 $\text{Var}(r)_m$ is large, it indicates strong divergence in gain or risk among candidate paths in the mode,
 839 demonstrating that the mode itself is not highly stable.

840 **Mode Radius.** The mode radius is defined as the average distance from all candidate paths within
 841 the mode to the representative path:

$$\rho_m = \frac{1}{|\mathcal{M}_t^{(m)}|} \sum_{\tau_k \in \mathcal{M}_t^{(m)}} d(\tau_k, \tilde{\tau}_t^{(m)}), \quad (78)$$

842 where $\tilde{\tau}_t^{(m)}$ is the representative path of the mode, and $d(\cdot, \cdot)$ is the path distance defined in Appendix
 843 B.4.2. ρ_m measures the compactness of the mode in the path space: a smaller ρ_m implies that
 844 internal paths are closer to each other and structurally more concentrated; a larger ρ_m implies that the
 845 candidate paths covered by the mode are more dispersed.

846 In summary, the mode statistics $\chi_t^{(m)}$ compactly describe the exploration mode via scale, mean,
 847 variance, and dispersion. It preserves not only the overall gain and risk attributes of the mode but
 848 also characterizes its internal consistency and stability, thereby providing structured input for the
 849 subsequent mode scorer.

B.4.4 Planning and Mode Construction Details

This section details the explicit planner $Plan(\cdot)$, the path clustering algorithm, the representative path construction method, and how the number of modes M_t is determined.

Planner Implementation. Given the current pose c_t , anchor $a_t^{(k)}$, and obstacle map $\hat{O}_t$, the planner

$$\tau_t^{(k)} = Plan(c_t, a_t^{(k)}, \hat{O}_t) \quad (79)$$

is implemented as an A*-based 2D shortest path planner. Specifically, planning is performed on a discrete grid map corresponding to the predicted obstacle map, utilizing 8-neighbor connectivity. Grids with an obstacle probability higher than the threshold δ_{obs} are treated as impassable cells. The path cost is given by the Euclidean movement distance, employing a cost of $\sqrt{2}$ for diagonal movements. When the start point and the anchor endpoint fall within the same connected free region, A* directly returns the shortest path; otherwise, the path corresponding to this anchor is deemed infeasible and is filtered out during subsequent candidate set construction.

Path Clustering. After obtaining the candidate path set

$$\mathcal{T}_t = \{\tau_t^{(1)}, \dots, \tau_t^{(K)}\}, \quad (80)$$

we perform clustering in the path metric space induced by the path distance $d(\tau_i, \tau_j)$. In terms of implementation, we adopt agglomerative clustering based on a distance threshold δ_{mode} . This strategy does not pre-fix the number of clusters but adaptively determines the clustering results based on the distribution of current candidate paths. Therefore, the number of modes M_t is not a fixed hyperparameter but is adaptively determined by the geometric and topological distribution of the path set at the current time step:

$$\mathcal{T}_t \longrightarrow \{\mathcal{M}_t^{(1)}, \dots, \mathcal{M}_t^{(M_t)}\}. \quad (81)$$

Representative Path Construction. For each clustered mode $\mathcal{M}_t^{(m)}$, we define its representative path as the medoid within that cluster:

$$\tilde{\tau}_t^{(m)} = \arg \min_{\tau_i \in \mathcal{M}_t^{(m)}} \sum_{\tau_j \in \mathcal{M}_t^{(m)}} d(\tau_i, \tau_j). \quad (82)$$

This selection ensures that the representative path is always a truly executable candidate path, rather than an abstract average trajectory, making it more suitable as an input to the subsequent mode scorer. In implementation, if a mode contains only a single path, that path directly serves as the representative path for that mode. If the number of candidate paths is insufficient to form multiple valid clusters, the clustering process degenerates to a single-mode case, where $M_t = 1$.

B.5 Supervision Targets for Mode Scoring and Local Reranking

This appendix corresponds to Sec. 3.4 of the main text. The mode-level scoring heads do not directly calculate the total score via hand-crafted analytical formulas but rather output structured values produced by a learnable network. The overall pipeline is: 1. Construct a unified representation $z_t^{(m)}$ from mode input features; 2. Output 5 independent scoring heads via the mode scoring network s_η ; 3. Provide exclusive supervision signals or proxy targets for each scoring head during the training phase; 4. Combine the 5 scoring heads as a weighted sum to form the final mode score during the inference phase.

Strictly speaking, the mode scoring head is defined as:

$$(G_m, R_m, U_m, C_m, D_m) = s_\eta(z_t^{(m)}), \quad (83)$$

where $s_\eta(\cdot)$ is a neural network parameterized by η , rather than a pure mathematical analytical expression. To ensure the model is trainable and the results interpretable, we define clear supervision targets for each scoring head.

B.5.1 Mode Representation and Scorer Implementation

The mode representation needs to fuse the global mapping state, mode geometric features, and statistical attributes, defined as:

$$z_t^{(m)} = [\text{Pool}(z_t), \text{Enc}(\tilde{\tau}_t^{(m)}), \chi_t^{(m)}], \quad (84)$$

where: $\text{Pool}(z_t)$: the global context feature, obtained by applying global average pooling to z_t outputted by the difficulty-invariant encoder in Sec. 3.1; $\text{Enc}(\tilde{\tau}_t^{(m)})$: the representative path encoding,

obtained by passing the mode representative path $\tilde{\tau}_t^{(m)}$ through a lightweight sequence encoder after resampling; $\chi_t^{(m)}$: the mode statistics, characterizing the overall attributes of the mode, specifically defined in Sec. B.4.3.

In terms of implementation, we adopt the following specific forms. First, apply global average pooling to the shared context feature:

$$\bar{z}_t = \text{GAP}(z_t). \quad (85)$$

Subsequently, resample the representative path $\tilde{\tau}_t^{(m)}$ into a path sequence of fixed length S_m , and represent each path point as:

$$\tilde{\mathbf{p}}_{m,s} = [x_{m,s}, y_{m,s}, \sin \theta_{m,s}, \cos \theta_{m,s}], \quad s = 1, \dots, S_m. \quad (86)$$

Pass this sequence into a lightweight path encoder (implemented as an MLP) to obtain the mode path embedding:

$$e_t^{(m)} = \text{Enc}(\tilde{\tau}_t^{(m)}). \quad (87)$$

Finally, the mode representation is implemented as:

$$z_t^{(m)} = \text{Concat} \left(\text{MLP}_{\text{ctx}}(\bar{z}_t), \text{MLP}_{\text{path}}(e_t^{(m)}), \text{LN}(\chi_t^{(m)}) \right), \quad (88)$$

where $\text{LN}(\cdot)$ denotes Layer Normalization applied to the statistics.

Based on this representation, the mode scorer s_η consists of a shared backbone and 5 independent heads:

$$h_t^{(m)} = \text{MLP}_{\text{mode}}(z_t^{(m)}), \quad (89)$$

$$G_m = \text{Head}_G(h_t^{(m)}), \quad R_m = \text{Head}_R(h_t^{(m)}), \quad U_m = \text{Head}_U(h_t^{(m)}), \quad (90)$$

$$C_m = \text{Head}_C(h_t^{(m)}), \quad D_m = \text{Head}_D(h_t^{(m)}). \quad (91)$$

Here, each $\text{Head}_\bullet$ outputs a scalar scoring head. The shared backbone is utilized to model the overall attributes of the mode, while the multi-head outputs respectively characterize gain, risk, uncertainty, cost, and diversity.

B.5.2 Supervision Targets for Mode Scoring

Expected Mode Gain G_m . In the main text, G_m represents the learnable gain output head of the mode scorer; its training supervision target is:

$$G_m^{\text{GT}} = \max_{\tau \in \mathcal{M}_t^{(m)}} \Delta \text{Cov}(\tau), \quad (92)$$

where $\Delta \text{Cov}(\tau)$ represents the actual accumulated surface coverage improvement obtained after executing along path τ . The maximum value is chosen instead of the average value because mode-level selection is more concerned with “whether this mode contains a high-quality path worth executing,” i.e., the upper bound of the mode’s potential.

Mode Risk R_m . In the main text, R_m represents the learnable risk output head of the mode scorer; its training supervision target is:

$$R_m^{\text{GT}} = \frac{1}{|\mathcal{M}_t^{(m)}|} \sum_{\tau_k \in \mathcal{M}_t^{(m)}} r_k. \quad (93)$$

The average value is chosen here rather than the minimum or maximum because risk reflects the overall safety and executability of the mode, which must synthesize the statistical features of all candidate paths within the mode.

Mode Uncertainty U_m . In the main text, U_m represents the learnable uncertainty output head of the mode scorer; its training target (uncertainty proxy) is denoted as:

$$U_m^{\text{GT}} = U_m^{\text{disp}} + \beta U_t^{\text{diff}} U_m^{\text{aux}}, \quad (94)$$

where:

Intra-mode divergence

$$U_m^{\text{disp}} = \text{Var}_{\tau \in \mathcal{M}_t^{(m)}} (g(\tau)), \quad (95)$$

represents the dispersion degree of different candidate paths within the mode on the gain proxy. If this value is large, it indicates that path gains within the mode are unstable.

929 Consistency deviation from the auxiliary value map

$$U_m^{\text{aux}} = |\bar{g}_m - \bar{g}_m^{\text{aux}}|, \quad (96)$$

930 where $\bar{g}_m$ represents the average gain proxy of candidate paths within the mode, and $\bar{g}_m^{\text{aux}}$ represents
931 the mode-level gain estimation induced by the auxiliary value map $\hat{M}_t^{\text{aux}}$. If this deviation is large, it
932 indicates that the current mode is inconsistent with the dense exploration prior in the shared context.

933 Difficulty ambiguity signal

$$U_t^{\text{diff}} = \mathcal{H}(q_\omega^{\text{ood}}(d | z_t)), \quad (97)$$

934 where $\mathcal{H}(\cdot)$ is the entropy function, and $q_\omega^{\text{ood}}(d | z_t)$ is a non-adversarial OOD difficulty estimator.
935 This term characterizes the degree of ambiguity when the current context is located near the boundaries
936 of different difficulty distributions. Through the above definitions, U_m^{GT} provides supervision for the
937 uncertainty head of the mode scorer, while the U_m predicted by the network actually participates in
938 the mode scoring during inference.

939 **Mode Cost C_m .** In the main text, C_m represents the learnable cost output head of the mode scorer;
940 its training supervision target is:

$$C_m^{\text{GT}} = \alpha_c \cdot \text{len}(\tilde{\tau}_t^{(m)}) + (1 - \alpha_c) \cdot h(\tilde{\tau}_t^{(m)}, \bar{\tau}_{t-1}), \quad (98)$$

941 where: $\text{len}(\tilde{\tau}_t^{(m)})$: length of the mode’s representative path; $h(\tilde{\tau}_t^{(m)}, \bar{\tau}_{t-1})$: the continuity deviation
942 between the mode’s representative path and the path submitted for execution at the previous time step;
943 $\alpha_c \in [0, 1]$: a weight balancing the length cost and the switching cost. This target simultaneously
944 characterizes “how much geometric cost must be paid to follow this mode” and “how much decision
945 discontinuity will be introduced by switching to this mode.”

946 **Diversity Bonus D_m .** In the main text, D_m represents the learnable diversity output head of the
947 mode scorer; its training supervision target is:

$$D_m^{\text{GT}} = \min_{m' \neq m} d(\tilde{\tau}_t^{(m)}, \tilde{\tau}_t^{(m')}), \quad (99)$$

948 where $d(\cdot, \cdot)$ is the path distance metric defined in Appendix B.4.2. The intuitive meaning of this
949 definition is: if the representative path of a certain mode is far from all other modes, it indicates
950 that this mode corresponds to a more unique long-horizon exploration strategy, and therefore should
951 receive a higher diversity bonus. It is important to emphasize that this term is not intended to
952 encourage unconstrained mode dispersion, but rather to provide the selector with an auxiliary signal
953 preferring a “unique mode with more exploration value” when gains and risks are similar.

954 B.5.3 Local Reranker Representation and Supervision Targets

955 This section corresponds to the path-level representation of the local reranker in the main text:

$$\zeta_t(\tau) = \Lambda(z_t, \tau, \varphi(\tau), \chi_t^{(m_t^*)}), \quad (100)$$

956 and its output heads:

$$\hat{G}(\tau), \hat{R}(\tau), \hat{C}(\tau), \hat{H}(\tau). \quad (101)$$

957 **Path-Level Representation and Fusion Function.** The goal of the local reranker is to select
958 the trajectory most suitable for actual execution from multiple candidate paths within the chosen
959 mode. To achieve this, we need to fuse four types of information simultaneously: Shared context:
960 global mapping information from the current state encoding; Path geometry: the geometric shape and
961 orientation changes of the candidate path itself; Path-level structured descriptors: $\varphi(\tau)$ defined in
962 Appendix B.4.1; Mode-level statistical information: the overall attributes of the currently selected
963 mode $\chi_t^{(m_t^*)}$. Therefore, we adopt the following specific implementation:

$$\zeta_t(\tau) = \text{Concat} \left(\text{MLP}_{\text{ctx}}(\text{GAP}(z_t)), \text{MLP}_{\text{path}}(\text{Enc}(\tau)), \text{LN}(\varphi(\tau)), \text{LN}(\chi_t^{(m_t^*)}) \right), \quad (102)$$

964 where: $\text{Enc}(\tau)$: sequence encoding of the candidate path τ after resampling; $\text{MLP}_{\text{ctx}}(\cdot)$: maps
965 the shared context to the feature space required by the local reranker; $\text{MLP}_{\text{path}}(\cdot)$: maps the path
966 sequence encoding to a fixed-length embedding; $\text{LN}(\cdot)$: normalizes the explicit statistical features to
967 reduce training instability caused by scale differences. Based on the above path-level representation,
968 the local reranker employs a structure comprising a two-layer MLP backbone and four independent
969 heads:

$$h_t(\tau) = \text{MLP}_{\text{local}}(\zeta_t(\tau)), \quad (103)$$

$$\hat{G}(\tau) = \text{Head}_{\hat{G}}(h_t(\tau)), \quad \hat{R}(\tau) = \text{Head}_{\hat{R}}(h_t(\tau)), \quad (104)$$

$$\hat{C}(\tau) = \text{Head}_{\hat{C}}(h_t(\tau)), \quad \hat{H}(\tau) = \text{Head}_{\hat{H}}(h_t(\tau)). \quad (105)$$

Supervision Targets. For any path τ , we define the following supervision targets:

$$\hat{G}^{\text{GT}}(\tau) = \Delta \text{Cov}(\tau), \quad (106)$$

$$\hat{R}^{\text{GT}}(\tau) = r(\tau), \quad (107)$$

$$\hat{C}^{\text{GT}}(\tau) = c(\tau), \quad (108)$$

$$\hat{H}^{\text{GT}}(\tau) = h(\tau), \quad (109)$$

where $r(\tau)$, $c(\tau)$, and $h(\tau)$ are respectively induced by the path-level structured descriptors in Appendix B.4.1. Among these, the continuity term $\hat{H}(\tau)$ is particularly critical. In open environments, if exploration decisions change direction too frequently, even if every step is locally reasonable, it will lead to path oscillations, repeated visits, and efficiency loss overall. Therefore, the local reranker must not only find paths that “look highly profitable” within the mode but also favor trajectories that are more consistent with the current execution trend and can reduce high-frequency mode switching. In other words, $\hat{H}(\tau)$ assumes the role of a closed-loop stabilizer here. Consequently, the local reranker does not implicitly learn all information directly from the path, but rather conducts supervised learning based on the shared context and path geometry, combined with explicitly defined structured path attributes.

B.5.4 Training Objective of Hierarchical Selection

This section provides the complete training objectives for the hierarchical selection module, including the mode classification loss, the intra-mode ranking loss, and auxiliary supervision for the gain, risk, cost, continuity, and uncertainty heads. Let the oracle positive mode be m_t^+ , and the oracle optimal path within it be τ_t^+ . First, we employ a mode classification loss to supervise the mode-level selection:

$$\mathcal{L}_{\text{mode}} = -\log \frac{\exp(S_{m_t^+}^{\text{mode}})}{\sum_{m=1}^{M_t} \exp(S_m^{\text{mode}})}. \quad (110)$$

Subsequently, within the selected positive mode $\mathcal{M}_t^{(m_t^+)}$, we adopt a listwise ranking loss to train the local reranker:

$$\mathcal{L}_{\text{rank}} = -\log \frac{\exp(S_{\tau_t^+}^{\text{local}})}{\sum_{\tau \in \mathcal{M}_t^{(m_t^+)}} \exp(S_{\tau}^{\text{local}})}. \quad (111)$$

Furthermore, we impose auxiliary supervision on the multi-head outputs of the mode scorer and local reranker, where $\mathcal{L}_{\text{gain}}$, $\mathcal{L}_{\text{risk}}$, $\mathcal{L}_{\text{cost}}$, $\mathcal{L}_{\text{cont}}$, and $\mathcal{L}_{\text{unc}}$ respectively correspond to the gain, risk, cost, continuity, and uncertainty heads. Synthesizing all the above components, the overall objective of the hierarchical selection module is written as:

$$\mathcal{L}_{\text{sel}} = \lambda_{\text{mode}} \mathcal{L}_{\text{mode}} + \lambda_{\text{rank}} \mathcal{L}_{\text{rank}} + \lambda_{\text{gain}} \mathcal{L}_{\text{gain}} + \lambda_{\text{risk}} \mathcal{L}_{\text{risk}} + \lambda_{\text{cost}} \mathcal{L}_{\text{cost}} + \lambda_{\text{cont}} \mathcal{L}_{\text{cont}} + \lambda_{\text{unc}} \mathcal{L}_{\text{unc}}. \quad (112)$$

Where each auxiliary term is defined as:

$$\mathcal{L}_{\text{gain}} = \sum_{m=1}^{M_t} \|G_m - G_m^{\text{GT}}\|_2^2 + \|\hat{G}(\tau_t^+) - \hat{G}^{\text{GT}}(\tau_t^+)\|_2^2, \quad (113)$$

$$\mathcal{L}_{\text{risk}} = \sum_{m=1}^{M_t} \|R_m - R_m^{\text{GT}}\|_2^2 + \|\hat{R}(\tau_t^+) - \hat{R}^{\text{GT}}(\tau_t^+)\|_2^2, \quad (114)$$

$$\mathcal{L}_{\text{cost}} = \sum_{m=1}^{M_t} \|C_m - C_m^{\text{GT}}\|_2^2 + \|\hat{C}(\tau_t^+) - \hat{C}^{\text{GT}}(\tau_t^+)\|_2^2, \quad (115)$$

$$\mathcal{L}_{\text{cont}} = \|\hat{H}(\tau_t^+) - \hat{H}^{\text{GT}}(\tau_t^+)\|_2^2, \quad (116)$$

$$\mathcal{L}_{\text{unc}} = \sum_{m=1}^{M_t} \|U_m - U_m^{\text{GT}}\|_2^2. \quad (117)$$

Table 6: **Generalization results on GLEAM-Bench.** GLEAM-Bench evaluates active mapping generalization on 128 unseen indoor scenes drawn from both synthetic and real-scan datasets. Following [9], all learnable methods are trained on 1,024 training scenes and evaluated without fine-tuning using shared random initial poses.

Method	Overall			ProcTHOR & HSSD			Gibson & Matterport3D		
	Cov. $\uparrow$	AUC $\uparrow$	CD $\downarrow$	Cov. $\uparrow$	AUC $\uparrow$	CD $\downarrow$	Cov. $\uparrow$	AUC $\uparrow$	CD $\downarrow$
Random	31.41%	27.58%	2.15m	34.88%	30.85%	1.85m	20.89%	24.30%	2.76m
Vacuum [25]	42.96%	35.89%	1.67m	45.81%	37.56%	1.41m	37.13%	32.48%	2.20m
ANS [27]	48.86%	41.98%	1.39m	54.31%	46.93%	1.01m	37.71%	31.86%	2.17m
UPEN [3]	49.65%	42.98%	1.38m	54.84%	46.65%	1.01m	39.02%	35.47%	2.14m
OccAnt [6]	53.61%	46.03%	1.16m	60.30%	51.99%	0.82m	39.91%	33.84%	1.85m
FBE [18]	56.80%	45.07%	0.94m	61.56%	50.99%	0.56m	46.17%	37.60%	1.72m
ANM [4]	57.01%	49.56%	1.02m	63.98%	56.37%	0.66m	42.73%	35.63%	1.77m
GLEAM [9]	66.50%	57.63%	0.80m	76.01%	66.13%	0.38m	47.04%	40.23%	1.67m
Ours	68.90%	58.04%	0.74m	77.01%	68.37%	0.30m	49.18%	42.80%	1.55m

C Supplementary Experiments

This section provides additional quantitative experiments, ablations, and hyperparameter analyses that complement the results in the main paper.

C.1 Extended Generalization Evaluation on GLEAM-Bench [9]

We further evaluate our method on GLEAM-Bench [9], a large-scale benchmark specifically designed for *generalizable* active mapping in complex indoor scenes. GLEAM-Bench contains 1,152 diverse 3D scenes from both synthetic and real-scan datasets, including ProcTHOR, HSSD, Gibson, and Matterport3D, with 1,024 scenes for training and 128 unseen scenes for evaluation. Its scenes are characterized by diverse floorplans, complex interconnectivity, and mixed domain sources, making the evaluation inherently out-of-distribution rather than a simple within-scene interpolation setting [9]. In particular, the inclusion of unseen real-scan scenes further increases the domain gap and provides a stronger test of generalization.

As shown in Table 6, our method achieves the best performance across all reported metrics. Compared with the previous SOTA GLEAM [9], our method improves Overall Coverage from 66.50% to 68.90%, AUC from 57.63% to 58.04%, and reduces Chamfer Distance from 0.80m to 0.74m. The gains remain consistent on both the synthetic subset (ProcTHOR & HSSD) and the more challenging real-scan subset (Gibson & Matterport3D). Notably, on Gibson & Matterport3D, our method improves Coverage from 47.04% to 49.18%, AUC from 40.23% to 42.80%, and reduces Chamfer Distance from 1.67m to 1.55m. These results suggest that the proposed multimodal long-horizon modeling improves not only exploration efficiency but also transferability across heterogeneous scene distributions, further supporting our central claim on OOD generalization.

C.2 Why Deterministic Single-Goal Prediction Limits Generalization

We next examine whether the weak generalization of existing methods is fundamentally related to deterministic single-goal prediction. To this end, we adopt a unified out-of-distribution protocol throughout this subsection: *all variants are trained on AiMDoom and directly tested on MP3D without fine-tuning*. This setting is intentionally challenging and evaluates whether the learned long-horizon decision mechanism can transfer across datasets.

C.2.1 Comparison of proposal generation paradigms.

We first compare different long-horizon proposal generation strategies under the same downstream planning and selection pipeline. For a fair comparison, all variants share the same state representation, context encoder, obstacle-aware planner, exploration-mode clustering, and downstream selector, and differ only in the proposal generation module. Specifically, *Single-anchor Regression* replaces our multimodal generator with a deterministic regression head that predicts a single coarse exploration anchor from the current context feature. *Diffusion Anchor Generator* replaces the proposal module with a conditional diffusion model defined in the same coarse anchor space. *Vanilla FM Anchor Generator* uses a standard conditional flow matching model in the same anchor space, but without our task-specific multimodal proposal formulation for active 3D mapping. *Ours (Task-Specific CFM)* uses

Table 7: Comparison of long-horizon proposal generation strategies.

Method	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$
Single-anchor Regression	62.47	10.04
Diffusion Anchor Generator	66.31	9.98
Vanilla FM Anchor Generator	67.83	9.78
Ours (Task-Specific CFM)	68.45	9.11

our task-specific conditional flow matching design to model a conditional distribution over coarse exploration anchors tailored to multimodal long-horizon exploration under partial observability.

As shown in Table 7, *Single-anchor Regression* performs the worst among all variants, confirming that deterministic single-point prediction is insufficient for open-environment active 3D mapping, where the same partial observation may correspond to multiple plausible long-horizon exploration directions. Replacing deterministic regression with generative anchor proposal modeling consistently improves cross-dataset transfer performance. In particular, the completion ratio increases from 62.47 to 66.31 and 67.83 when switching from deterministic regression to diffusion-based and vanilla flow-matching-based proposal generation, respectively, while the completion error is reduced from 10.04 to 9.98 and 9.78. This trend indicates that multimodal long-horizon hypothesis generation is better aligned with the underlying decision structure of the task.

Among the generative baselines, both *Diffusion Anchor Generator* and *Vanilla FM Anchor Generator* outperform single-anchor regression, but still underperform our full model. In particular, our method achieves the best completion ratio and the lowest completion error, reaching 68.45 in Comp. (%) and 9.11 in Comp. (cm). Compared with the strongest generic generative baseline, *Vanilla FM Anchor Generator*, our method further improves Comp. (%) from 67.83 to 68.45 and reduces Comp. (cm) from 9.78 to 9.11. These results suggest that the gain does not come from generative modeling alone, but from task-specifically adapting CFM to coarse long-horizon anchor generation.

C.2.2 Can post-hoc multi-goal extension replace explicit multimodal modeling?

While the above comparison shows that multimodal proposal generation is more effective than deterministic single-anchor regression, one may still ask whether the gain can be reproduced by simply extracting multiple high-value goals from a single-goal value map. To answer this question, we construct a stronger post-hoc multi-goal baseline based on NBP [8]. Recall that NBP predicts a value map and an obstacle map, and derives a single long-term goal from the maximum of the value map during inference. We replace this single-goal extraction step with a top- k candidate selection scheme: we first extract the top- k highest-scoring goal candidates from the predicted value map, then plan one candidate path for each goal using the same obstacle-aware planner as NBP, and finally select one executable path according to the same score-based decision rule. Importantly, this modification does not alter the NBP training objective, network architecture, or supervision, and only changes the inference-time goal extraction strategy.

As shown in Table 8, post-hoc multi-goal extension provides only limited improvement over the original NBP baseline. Specifically, increasing the number of candidate goals from one to three and five improves the completion ratio only marginally, from 61.81 to 62.27 and 62.54, while reducing the completion error from 10.22 to 10.07 and 9.93. Although these gains indicate that considering more candidate goals is somewhat beneficial, the improvement remains much smaller than that of our full model. In contrast, our method reaches 68.45 in Comp. (%) and 9.11 in Comp. (cm), substantially outperforming both NBP and its top- k variants. This result suggests that our advantage does not come merely from enumerating more candidate goals at inference time. Instead, it comes from explicitly learning a multimodal long-horizon proposal distribution under partial observability, rather than post-hoc candidate expansion on top of a deterministic single-goal predictor.

C.2.3 Anchor-space CFM vs. trajectory-space CFM

We further validate the task-specific design choice behind our method. As discussed in Sec. 3.2, active 3D mapping fundamentally requires modeling *long-horizon exploration intent* under partial observability, rather than directly synthesizing full trajectories. To empirically support this point, we compare our anchor-based formulation with a direct trajectory-generation baseline under the same cross-dataset OOD protocol: all variants are trained on AiMDoom and directly tested on MP3D without fine-tuning.

Table 8: Ablation on post-hoc multi-goal extension of NBP under the cross-dataset setting.

Method	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$
NBP	61.81	10.22
NBP + Top-3	62.27	10.07
NBP + Top-5	62.54	9.93
Ours	68.45	9.11

Table 9: Anchor-space CFM vs. trajectory-space CFM.

Metrics	Trajectory-space CFM	Anchor-space CFM (Ours)
Comp. (%) $\uparrow$	67.28	68.45
Comp. (cm) $\downarrow$	9.67	9.11
Convergence Epoch $\downarrow$	12.4	8.6
Distinct Modes@K $\uparrow$	2.1	3.4
Executable Proposals (%) $\uparrow$	81.5	96.2

For a fair comparison, both variants use the same state representation, context encoder, CFM backbone, sampling budget K , and downstream clustering/selection budget. They differ only in the proposal space. *Trajectory-space CFM* directly models a conditional distribution over fixed-length waypoint trajectories in a high-dimensional path space. *Anchor-space CFM (Ours)* instead models a conditional distribution over coarse long-horizon anchors in a low-dimensional anchor space, and then recovers executable candidate paths through obstacle-aware planning.

To match the three claims in Sec. 3.2, we report four groups of metrics. *Comp. (%)* and *Comp. (cm)* evaluate final mapping performance. *Convergence Epoch* measures optimization difficulty, where a smaller value indicates faster convergence. *Convergence Epoch* is defined as the first training epoch at which the validation performance reaches 95% of the final best result. *Distinct Modes@K* measures how many non-redundant exploration modes are preserved among the K generated proposals. *Executable Proposals (%)* denotes the fraction of generated proposals that remain collision-free and reachable after obstacle-based feasibility checking.

Table 9 supports all three claims. First, operating in anchor space is easier to optimize: compared with *Trajectory-space CFM*, our method converges substantially faster, reducing the convergence epoch from 12.4 to 8.6. This is consistent with the intuition that learning a conditional distribution over low-dimensional exploration intents is easier than directly modeling high-dimensional full trajectories.

Second, anchor-space modeling preserves multimodal long-horizon hypotheses more effectively. Although both methods are multimodal, directly generating full trajectories produces more redundant proposals, whereas our formulation retains a larger number of distinct exploration modes. In the results above, Distinct Modes@K increases from 2.1 to 3.4, indicating that the anchor-based formulation better captures diverse long-horizon futures under the same proposal budget.

Third, anchor-space CFM is naturally more compatible with explicit planning. Because our method predicts coarse goals and lets the planner impose obstacle-aware geometric realization, the generated proposals are far more likely to remain executable. In contrast, trajectory-space generation must implicitly learn both high-level exploration intent and low-level geometric feasibility in the same output space, leading to a lower executable-proposal ratio. As a result, our method also achieves better final OOD performance, improving Comp. (%) from 67.28 to 68.45 and reducing Comp. (cm) from 9.67 to 9.11.

C.3 Hyperparameter Settings and Sensitivity Analysis

This appendix reports the default hyperparameter settings and sensitivity analyses of several key design parameters in our framework. We focus on three factors that are most directly related to our method design: the number of sampled anchors K , the number of CFM sampling integration steps, and the clustering granularity controlled by the mode-clustering threshold δ_{mode} .

C.3.1 Default Hyperparameter Settings

Table 10 summarizes the default hyperparameter settings used in our final model. We choose a moderate anchor number and a moderate CFM integration budget to balance multimodal coverage,

Table 10: **Default hyperparameter settings of the proposed method.**

Category	Hyperparameter	Default Value
State representation	Number of height slices K_{slice}	4
Anchor generation	Number of sampled anchors K	5
Anchor generation	CFM sampling steps steps	16
Anchor generation	Noise scale temperature	1.0
Anchor generation	Diversity bandwidth σ	0.35
Anchor generation	Diversity weight λ_{div}	0.05
Planning	Obstacle threshold δ_{obs}	0.50
Mode construction	Clustering threshold δ_{mode}	0.45
Mode construction	Path-distance weights $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$	(0.35, 0.25, 0.25, 0.15)
Selection	Auxiliary selector weight $\lambda_{\text{aux}}^{\text{sel}}$	0.50
Local reranking	$(\lambda_R, \lambda_C, \lambda_H)$	(1.0, 0.5, 0.5)
Inference	Commitment length L_c	5

Table 11: **Sensitivity to the number of sampled anchors K .** Metrics are reported under the cross-dataset OOD setting.

K	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$	Relative inference cost
1	64.92	9.82	1.00 $\times$
3	67.71	9.29	1.31 $\times$
5	68.45	9.11	1.58 $\times$
8	68.34	9.15	2.07 $\times$

sampling quality, and computational cost. For mode construction, we use agglomerative clustering with threshold δ_{mode} , while the number of modes M_t is adaptively determined rather than pre-fixed. For readability, we report only the main default values that are most directly related to proposal generation, mode construction, and downstream selection.

C.3.2 Sensitivity to the Number of Sampled Anchors K

The number of sampled anchors K directly controls the expressiveness of multimodal long-horizon proposal generation. If K is too small, the model may fail to cover multiple plausible exploration directions. If K is too large, the candidate set becomes increasingly redundant and introduces additional planning and selection cost.

Table 11 reports the sensitivity to K . We observe that increasing K from 1 to 5 consistently improves both completion ratio and completion error, showing that explicit multimodal proposal generation is beneficial for OOD transfer. However, the gain saturates beyond $K = 5$, while the computational cost continues to grow. Therefore, we use $K = 5$ as the default setting.

C.3.3 Sensitivity to the Number of CFM Sampling Steps

We next analyze the effect of the number of Euler integration steps used during CFM sampling. A smaller number of steps leads to faster inference but coarser transport dynamics, while a larger number of steps yields more accurate generation at a higher computational cost. Following the implementation in Appendix B.3.4, the Euler step size is $\Delta t = 1/\text{steps}$.

It can be seen from the Table 12 that the performance keeps improving when the sampling steps increase from 4 to 16, while the gain becomes very limited when the steps are further raised from 16 to 32. Considering that 32 steps will significantly increase the inference cost, we finally select 16 as the default setting to achieve a better balance between performance and efficiency.

C.3.4 Sensitivity to Clustering Granularity δ_{mode}

Since our method explicitly clusters candidate paths into exploration modes, the clustering granularity also affects performance. If the threshold δ_{mode} is too small, the path set is over-fragmented into too many modes, weakening the benefit of redundancy reduction. If δ_{mode} is too large, semantically distinct trajectories may be merged too aggressively, reducing the diversity of exploration modes.

Table 12: **Sensitivity to the number of CFM sampling steps**. Metrics are reported under the cross-dataset OOD setting.

steps	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$	Relative inference cost
4	67.58	9.36	$0.58\times$
8	68.12	9.22	$0.79\times$
16	68.45	9.11	$1.00\times$
32	68.49	9.08	$1.71\times$

Table 13: **Sensitivity to clustering granularity controlled by δ_{mode}** . We vary the agglomerative clustering threshold under the cross-dataset OOD setting.

δ_{mode}	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$	Avg. realized modes
0.25	67.88	9.34	5.8
0.45	68.45	9.11	3.4
0.65	68.07	9.24	2.1
0.85	67.31	9.47	1.3

Importantly, we do not directly tune the number of modes M_t , because in our implementation M_t is adaptively determined by agglomerative clustering and is therefore not a fixed hyperparameter.

Table 13 varies the clustering threshold δ_{mode} . We find that a moderate clustering granularity achieves the best trade-off: too small a threshold leads to redundant modes, whereas too large a threshold over-merges distinct proposals. This result further supports the role of exploration-mode clustering as a meaningful intermediate abstraction between raw candidate paths and hierarchical selection.

C.3.5 Sensitivity to Diversity Regularization Weight λ_{div}

We further study the effect of the diversity regularization weight λ_{div} , which controls how strongly the sampled anchors are encouraged to spread across distinct long-horizon exploration modes. If λ_{div} is too small, the generated anchors may collapse to only a few dominant modes, reducing multimodal coverage. If it is too large, anchor dispersion may be overemphasized at the expense of proposal quality and downstream executability.

Table 14 varies λ_{div} under the same cross-dataset OOD protocol as the other sensitivity studies. We observe that a moderate diversity weight achieves the best trade-off. Removing the regularizer ($\lambda_{\text{div}} = 0$) reduces both completion quality and the number of realized distinct modes, indicating that explicit diversity encouragement is beneficial for multimodal long-horizon proposal generation. However, overly strong regularization also slightly degrades performance, likely because anchor dispersion starts to dominate over context-consistent proposal quality. Therefore, we use $\lambda_{\text{div}} = 0.05$ as the default setting.

C.4 Computational Cost

We report the computational cost of our framework to complement the main performance results. Unless otherwise stated, all measurements are obtained under the same implementation setting as the main experiments, using two 48GB GPUs, an effective batch size of 448, and at most 15 training iterations with early stopping.

Our full model requires approximately 30 hours of training to converge under the default setting. Compared with the deterministic NBP baseline, the additional computation mainly comes from multimodal anchor generation, candidate path construction, and the subsequent clustering and hierarchical selection stages. However, since anchor generation is performed in a low-dimensional anchor space, the overall overhead remains moderate in practice.

Although our method is moderately more expensive than NBP in both memory usage and inference latency, the additional cost is associated with explicit multimodal long-horizon modeling, which substantially improves generalization under both difficulty shift and cross-dataset transfer.

Table 14: **Sensitivity to the diversity regularization weight λ_{div} .** Metrics are reported under the cross-dataset OOD setting. Distinct Modes@K denotes the average number of non-redundant exploration modes realized among the sampled proposals.

λ_{div}	Comp. (%) $\uparrow$	Comp. (cm) $\downarrow$	Distinct Modes@K $\uparrow$
0.00	67.74	9.43	2.3
0.02	68.19	9.24	2.9
0.05	68.45	9.11	3.4
0.10	68.11	9.27	3.8

Table 15: **Computational cost comparison.** The NBP parameter count is measured from the released implementation.

Metric	NBP [8]	Ours
Training hardware	$1 \times 48\text{GB GPUs}$	$2 \times 48\text{GB GPUs}$
Effective batch size	56	56
Max training iterations	15	15
Training time	26 h	~ 30 h
Parameter count	49.96M	56.8M
Peak training memory	42.4 GB	53.8 GB
Inference time / decision step	64 ms	112 ms
Proposal generation	–	34 ms
Planning + selection	–	78 ms

D Supplementary Visualizations and Failure Analysis

D.1 Additional Qualitative Comparison with NBP.

Fig. 5 provides more qualitative comparisons between our method and NBP on representative scenes. Starting from the same initial condition, NBP tends to commit early to a single exploration direction and leaves some informative branches insufficiently explored, as highlighted by the dashed purple boxes. In contrast, our method achieves more complete coverage in these regions, which is consistent with our main claim that multimodal long-horizon modeling leads to more robust exploration and better generalization.

D.2 Failure Cases and Limitations

Fig. 6 presents representative failure cases of our method. Although our framework improves long-horizon exploration and generalization in open environments, it may still miss some informative regions, especially those highlighted by the dashed purple boxes.

We observe two typical failure patterns. First, when an unexplored region is only weakly connected to the current explored area, the generated anchors and downstream planner may still prefer nearer and more accessible branches, delaying exploration of distant but informative regions. Second, in scenes with ambiguous topology, narrow passages, or weak visual connectivity, mode construction and hierarchical selection may favor locally consistent hypotheses, leaving some isolated subspaces insufficiently explored.

These cases also reveal several limitations of the current framework. Since our method models multimodal exploration in a coarse anchor space rather than directly reasoning over full future map completion, the utility of distant but weakly indicated regions may not always be captured well. In addition, the downstream planning and clustering stages depend on the quality of the predicted obstacle map and candidate-path structure, so errors in these intermediate representations can affect mode construction and final selection. Future work may benefit from stronger global uncertainty estimation, more topology-aware proposal generation, and tighter coupling between multimodal hypothesis generation and downstream planning.

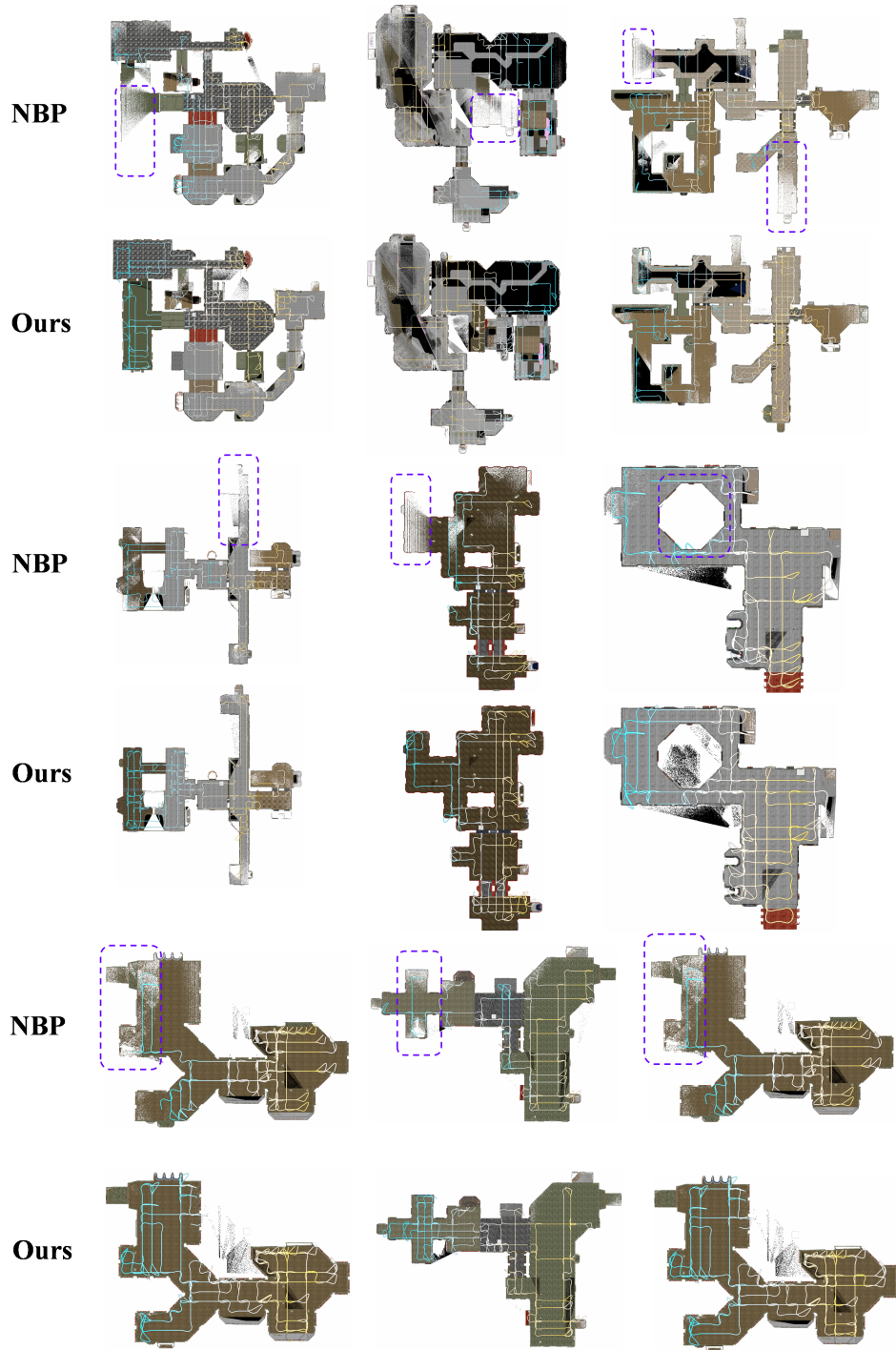


Figure 5: **Additional qualitative comparison with NBP.** Each column shows one representative scene, with NBP on the top and our method on the bottom, starting from the same initial condition. The dashed purple boxes highlight regions that are insufficiently explored by NBP but are more completely covered by our method. Compared with NBP, our method better preserves informative exploration hypotheses and achieves more complete scene coverage in challenging layouts.

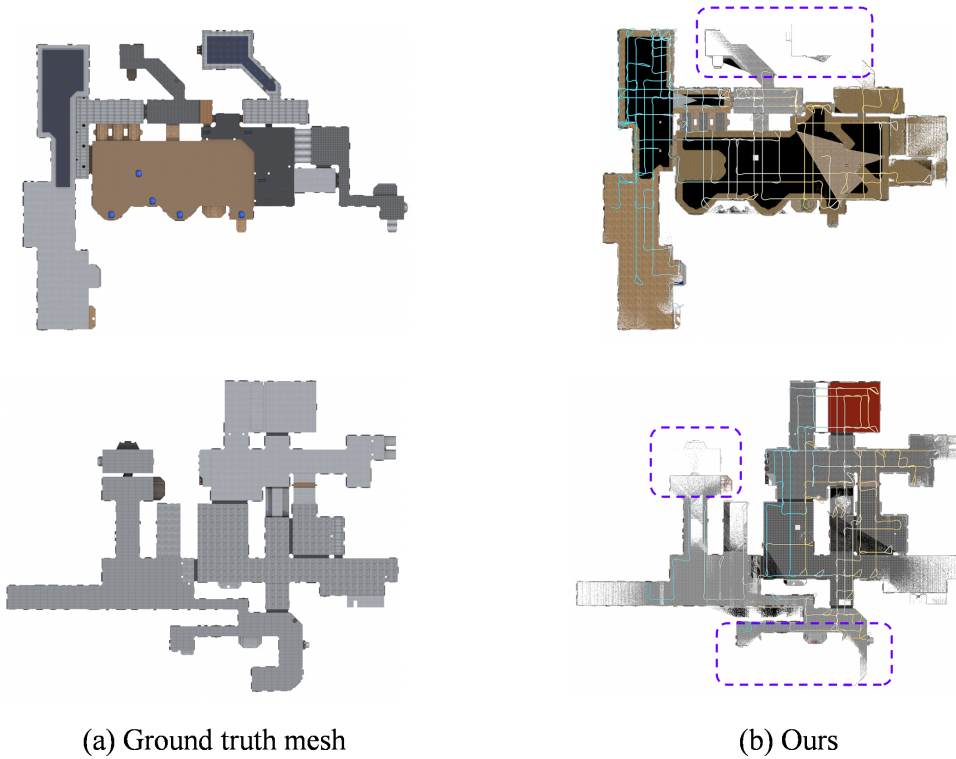


Figure 6: **Failure cases of our method.** The left column shows the ground-truth mesh, and the right column shows the reconstructed map and executed exploration trajectories of our method. The dashed purple boxes highlight missed or insufficiently explored regions. These cases typically occur when informative regions are weakly connected to the explored area or when the scene topology is highly ambiguous, causing the agent to favor locally consistent but less informative branches.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: We clarify the research contributions and scope of the study in the abstract and introduction of the paper.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Failure cases and limitations are discussed in Appendix ??.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: Yes. The paper includes theoretical analysis in Sec. 3.2, where Proposition 3.1 and Proposition 3.2 justify the formulation of multimodal anchor modeling and the use of CFM. Full proofs are given in Appendix B.1.1 and Appendix B.1.2.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We detail the model network structure and hyperparameter configurations needed to reproduce the results in the Experiments and Appendix sections.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

1317 Question: Does the paper provide open access to the data and code, with sufficient instruc-
1318 tions to faithfully reproduce the main experimental results, as described in supplemental
1319 material?

1320 Answer: [Yes]

1321 Justification: The code will be released upon acceptance. The datasets used derive from
1322 AimDooM and MP3d; they are already public and available online.

1323 Guidelines:

- 1324 • The answer NA means that paper does not include experiments requiring code.
- 1325 • Please see the NeurIPS code and data submission guidelines ([https://nips.cc/](https://nips.cc/public/guides/CodeSubmissionPolicy)
1326 [public/guides/CodeSubmissionPolicy](https://nips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 1327 • While we encourage the release of code and data, we understand that this might not be
1328 possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not
1329 including code, unless this is central to the contribution (e.g., for a new open-source
1330 benchmark).
- 1331 • The instructions should contain the exact command and environment needed to run to
1332 reproduce the results. See the NeurIPS code and data submission guidelines ([https://](https://nips.cc/public/guides/CodeSubmissionPolicy)
1333 nips.cc/public/guides/CodeSubmissionPolicy) for more details.
- 1334 • The authors should provide instructions on data access and preparation, including how
1335 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 1336 • The authors should provide scripts to reproduce all experimental results for the new
1337 proposed method and baselines. If only a subset of experiments are reproducible, they
1338 should state which ones are omitted from the script and why.
- 1339 • At submission time, to preserve anonymity, the authors should release anonymized
1340 versions (if applicable).
- 1341 • Providing as much information as possible in supplemental material (appended to the
1342 paper) is recommended, but including URLs to data and code is permitted.

1343 6. Experimental setting/details

1344 Question: Does the paper specify all the training and test details (e.g., data splits, hyper-
1345 parameters, how they were chosen, type of optimizer, etc.) necessary to understand the
1346 results?

1347 Answer: [Yes]

1348 Justification: The details necessary to faithfully reproduce our experiments (training proce-
1349 dure, data splits, optimizer, etc.) are included in the paper.

1350 Guidelines:

- 1351 • The answer NA means that the paper does not include experiments.
- 1352 • The experimental setting should be presented in the core of the paper to a level of detail
1353 that is necessary to appreciate the results and make sense of them.
- 1354 • The full details can be provided either with the code, in appendix, or as supplemental
1355 material.

1356 7. Experiment statistical significance

1357 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1358 information about the statistical significance of the experiments?

1359 Answer: [Yes]

1360 Justification: We report average results of multiple runs in our experimental section. Our
1361 paper does not report error bars.

1362 Guidelines:

- 1363 • The answer NA means that the paper does not include experiments.
- 1364 • The authors should answer "Yes" if the results are accompanied by error bars, confi-
1365 dence intervals, or statistical significance tests, at least for the experiments that support
1366 the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We describe the computational resource in Section C.4

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We reviewed and ensured that the present work respects the NeurIPS Code of Ethics at each individual part.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: Our paper is not highly related to societal impacts.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.

- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper does not pose any such risk.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All the datasets used to train our models, and the code for the papers utilized as benchmarks to evaluate our algorithms are cited.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: We do not release any new assets with this submission. However, we will make the code as well as trained models publicly available if/when the paper is accepted.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This research did not involve any crowdsourcing experiments or studies with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This research did not involve studies with human subjects

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.

- 1520 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1521 may be required for any human subjects research. If you obtained IRB approval, you
1522 should clearly state this in the paper.
- 1523 • We recognize that the procedures for this may vary significantly between institutions
1524 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1525 guidelines for their institution.
- 1526 • For initial submissions, do not include any information that would break anonymity (if
1527 applicable), such as the institution conducting the review.

1528 16. Declaration of LLM usage

1529 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1530 non-standard component of the core methods in this research? Note that if the LLM is used
1531 only for writing, editing, or formatting purposes and does not impact the core methodology,
1532 scientific rigorousness, or originality of the research, declaration is not required.

1533 Answer: [N/A]

1534 Justification: LLM is not an important, original, or non-standard component of the core
1535 methods in this research.

1536 Guidelines:

- 1537 • The answer NA means that the core method development in this research does not
1538 involve LLMs as any important, original, or non-standard components.
- 1539 • Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>)
1540 for what should or should not be described.